SĂPTĂMÂNA 10

Modele din date, AI + simulare și gemeni digitali

Modelare și Simulare · Anul III, Calculatoare și Tehnologia Informației

Ș.l. dr. ing. Cătălin Iapă

Unde am rămas

- ✓ Agenți: percepție, stare, decizie; comportamentul global **emerge** din interacțiuni (S9)
- ✓ Replicări: 60 de rulări ale epidemiei, vârful 1 164 ± 66 – o rulare nu ajunge
- ✓ Calibrare ABC: **400 de simulări** pentru un singur parametru
- ✓ Întrebarea de final: ce facem când o singură simulare durează o **oră**?

La finalul acestei săptămâni veți putea…

1. să construiți și să interpretați un **model de regresie** (coeficienți, Sy, r, R^2) și să distingeți antrenarea de testare
2. să recunoașteți **supraînvățarea** și să evaluați corect un model pe date nefolosite
3. să construiți un **model surogat** al unei simulări scumpe (plan LHS, proces gaussian) și să îl folosiți la calibrare
4. să explicați ideea rețelelor **informate de fizică** (PINN) și a descoperirii ecuațiilor din date (SINDy)
5. să descrieți folosirea simulării pentru **antrenarea AI**, decalajul sim-to-real și randomizarea domeniului
6. să definiți un **geamăn digital** și să implementați asimilarea datelor cu un filtru Kalman

Două feluri de a construi un model

Model mecanist (S1–S9)

- pornește de la **mecanisme**: sosiri, legi fizice, reguli
- parametrii au **sens**: λ, μ, k, β
- poate **extrapola** în situații nevăzute
- cere înțelegerea sistemului
- adesea **lent** de simulat

vs

Model din date (S10)

- pornește de la **observații**
- parametrii sunt coeficienți fără sens fizic
- bun doar în zona acoperită de date
- nu cere înțelegerea mecanismului
- **rapid** de evaluat după antrenare

Holban (cap. 8) numește modelele din date **modele corelaționale**: descriu relația statistică dintre variabile, fără să explice cauza. Cele două abordări nu se exclud – cele mai bune modele moderne le combină.

Unde intră datele într-un studiu de simulare

1 Modelarea intrărilor

Distribuții potrivite pe date: timpi de servire, sosiri (S6).

2 Calibrare

Parametri aleși ca simularea să reproducă datele: β, γ, p (S7, S9).

3 Validare

Comparăm ieșirile cu date nefolosite la construcție (S6).

4 Modele surogat

Învățăm din rulări ale simulării un model care o imită instantaneu.

5 Componente învățate

O parte a modelului (frecare, comportament) este învățată din date.

6 Asimilarea datelor

Senzorii corectează continuu starea modelului: gemenii digitali.

Primele trei roluri le-am văzut deja. Astăzi le adăugăm pe ultimele trei, în care învățarea automată și simularea lucrează împreună.

Cutie albă, cutie gri, cutie neagră

- **Cutie albă**: modelul este scris complet din legi și mecanisme – M/M/1, Lotka–Volterra, Boids
- **Cutie neagră**: modelul este învățat complet din date – regresie, rețele neurale, gradient boosting
- **Cutie gri**: structura vine din fizică, iar o parte (parametri sau termeni lipsă) este învățată din date
- Cu cât știm mai puțin despre mecanism, cu atât ne bazăm mai mult pe date – și invers
- Datele costă: o cutie neagră cere mult mai multe observații decât o cutie gri

R **Regula practică**

Folosiți toată fizica pe care o cunoașteți sigur și învățați din date doar ce lipsește. Modelul rezultat are nevoie de mai puține date și extrapolează mai bine.

Ora 1

De la regresie la modele surogat

1. Modele corelaționale: regresia (Holban, cap. 8)
2. Date reale: închirierile de biciclete din Londra
3. Supraînvățare și generalizare
4. Modele surogat pentru simulări scumpe – demo

Exemplul din curs (Holban, cap. 8)

Modele corelaționale: regresia

1. Metoda celor mai mici pătrate
2. Exemplul cu 8 puncte, refăcut pas cu pas
3. Eroarea standard și coeficientul de corelație
4. Două neconcordanțe în textul original

Ce este un model de regresie

O relație statistică între o variabilă de ieșire și variabile explicative

Holban: modelele corelaționale descriu **relațiile statistice** dintre variabilele unui sistem. Căutăm o funcție $y = f(x)$ care aproximează cât mai bine perechile observate (x_i, y_i), fără să pretindem că x este **cauza** lui y.

Holban, cap. 8; Law, Simulation Modeling and Analysis, cap. 6

- **Liniară simplă**: $y = a_0 + a_1 \cdot x$ – o dreaptă prin norul de puncte
- **Multiplă**: $y = a_0 + a_1 \cdot x_1 + \ldots + a_k \cdot x_k$ – mai multe variabile explicative
- **Neliniară**: polinoame, exponențiale, modele învățate (arbori, rețele)
- Criteriul de potrivire: **suma pătratelor erorilor** minimă (Gauss, Legendre, ~1805)

Metoda celor mai mici pătrate pentru o dreaptă

$$a_1 = (n\Sigma xy - \Sigma x \Sigma y) / (n\Sigma x^2 - (\Sigma x)^2) \qquad a_0 = \bar{y} - a_1 \cdot \bar{x}$$

$E(a_0, a_1)$	$\Sigma (y_i - a_0 - a_1 x_i)^2$: suma pătratelor erorilor, pe care o **minimizăm**
$\partial E/\partial a_0 = 0$, $\partial E/\partial a_1 = 0$	două ecuații liniare în a_0 și a_1 (**ecuațiile normale**), cu soluția de mai sus
$\bar{x}, \bar{y}$	mediile datelor: dreapta trece mereu prin **centrul de greutate** ($\bar{x}$, $\bar{y}$)

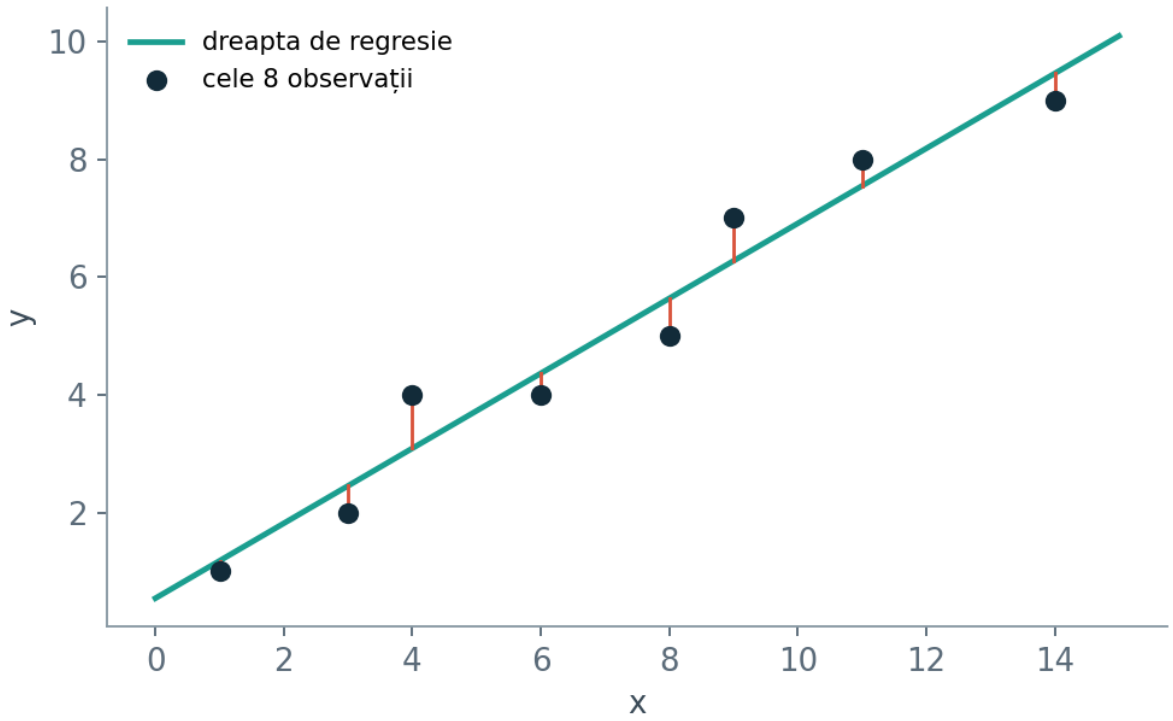
Pentru exemplul Holban: n = 8, Σx = 56, Σy = 40, Σxy = 364, Σx² = 524. Rezultă $a_1 = (8 \cdot 364 - 56 \cdot 40) / (8 \cdot 524 - 56^2) = 672 / 1\,056 = 0{,}6364$ și $a_0 = 5 - 0{,}6364 \cdot 7 = 0{,}5455$.

Exemplul Holban: 8 observații, valori calculate și erori

	1	3	4	6	8	9	11	14	Σ
x_i	1	3	4	6	8	9	11	14	56
y_i	1	2	4	4	5	7	8	9	40
$\hat{y}_i = a_o + a_1 x_i$	1,18	2,45	3,09	4,36	5,64	6,27	7,55	9,45	40
$(y_i - \hat{y}_i)^2$	0,033	0,207	0,826	0,132	0,405	0,529	0,207	0,207	2,545

- Dreapta: **$\hat{y} = 0{,}5455 + 0{,}6364 \cdot x$** ($a_o = 6/11$, $a_1 = 7/11$ exact)
- Suma valorilor calculate este egală cu suma valorilor observate (40) – o verificare rapidă a calculului
- Cea mai mare eroare este la x = 4: punctul (4; 4) este cel mai departe de dreaptă

Dreapta de regresie și erorile ei



liniile roșii sunt erorile; metoda minimizează suma pătratelor lor

- $\hat{y} = 0{,}55 + 0{,}64 \cdot x$
- Panta: fiecare unitate în plus de x adaugă în medie **0,64** unități la y
- Dreapta trece prin punctul mediilor (7; 5)
- Erorile pozitive și negative se compensează: suma lor este **zero**
- Pătratul penalizează mult erorile mari – un singur punct aberant poate înclina dreapta

Cât de bun este modelul: eroarea standard și coeficientul de corelație

$$Sy = \sqrt{\left(\Sigma (y_i - \hat{y}_i)^2 / n \right)} \qquad r = \sqrt{\left(1 - Sy^2 / sy^2 \right)}$$

Sy	eroarea standard a estimării: abaterea tipică față de dreaptă; aici √(2,545 / 8) = **0,564**
sy^2	dispersia lui y în jurul mediei: Σy²/n − ȳ² = 256/8 − 25 = **7**
r	coeficientul de corelație: 1 = potrivire perfectă, 0 = nicio relație liniară; aici **0,977**
$R^2 = r^2$	fracțiunea din variația lui y explicată de model: **95,5%**

În regresia liniară simplă, r calculat astfel este egal cu coeficientul de corelație Pearson dintre x și y (cu semnul pantei). Pentru modele neliniare folosim direct $R^2 = 1 - \Sigma(y - \hat{y})^2 / \Sigma(y - \bar{y})^2$.

Două neconcordanțe în textul original

- Tabelul erorilor începe cu **0,324** pentru primul punct; corect este $(1 - 1{,}18)^2$ = **0,0331**
- Suma pătratelor erorilor este 2,545, deci Sy = 0,564 (în text ≈ 0,6, o rotunjire acceptabilă)
- Textul dă **r = 0,99**; cu Sy = 0,6 rezultă $\sqrt{(1 - 0{,}36/7)}$ = 0,974, iar cu valoarea exactă **0,977**
- Verificare independentă: corelația Pearson calculată direct din date este tot 0,977
- Concluzia nu se schimbă – relația este puternic liniară –, dar r = 0,99 ar sugera o potrivire aproape perfectă

V **Exercițiu de verificare**

Recalculați întotdeauna exemplele numerice cu un program. Trei linii de NumPy (np.polyfit, np.corrcoef) găsesc ambele neconcordanțe.

Verificare

? Un model de regresie are **$R^2 = 0{,}95$** pe datele folosite la construcție. Ce putem afirma cu siguranță?

A x este cauza lui y

B modelul va avea $R^2 \approx 0{,}95$ și pe date noi

C 95% din variația lui y din aceste date este explicată de model

D 95% dintre puncte sunt exact pe dreaptă

Regresia multiplă: performanța relativă a procesoarelor

$$PRP = -55{,}9 + 0{,}0489 \cdot MYCT + 0{,}0153 \cdot MMIN + 0{,}0056 \cdot MMAX + 0{,}6410 \cdot CACH - 0{,}2700 \cdot CHMIN + 1{,}480 \cdot CHMAX$$

MYCT	durata ciclului mașinii (ns)
MMIN, MMAX	memoria principală minimă și maximă (KB)
CACH	memoria cache (KB)
CHMIN, CHMAX	numărul minim și maxim de canale de intrare–ieșire

Exemplul din curs: 209 configurații de procesoare din anii ’80 (Ein-Dor și Feldheim, 1987). Atenție la interpretare: coeficientul negativ al lui CHMIN nu înseamnă că mai multe canale scad performanța – variabilele sunt corelate între ele, iar un coeficient arată efectul la celelalte variabile constante.

Liniar în parametri, nu neapărat în variabile

- Regresia „liniară” cere liniaritate în **coeficienți**: $y = a_0 + a_1x + a_2x^2$ este tot o regresie liniară
- Transformări utile: log y pentru creșteri exponențiale, $1/(1 - \rho)$ pentru cozi (S4), variabile indicator pentru categorii
- Forma matriceală: $\mathbf{a = (X^TX)^{-1}X^Ty}$ – o singură linie în NumPy: np.linalg.lstsq(X, y)
- Variabile **indicator** (0/1): ora din zi, ziua săptămânii, sezonul – fiecare categorie primește propriul coeficient
- Limita: interacțiunile și pragurile trebuie inventate de noi; modelele învățate (arbori, rețele) le găsesc singure

! Corelație ≠ cauzalitate

Vânzările de înghețată și înecurile sunt corelate: ambele cresc vara. O regresie bună prezice, dar nu spune ce se întâmplă dacă intervenim – pentru asta avem nevoie de un model mecanist sau de un experiment.

Date reale (laboratorul 7)

Închirierile de biciclete din Londra

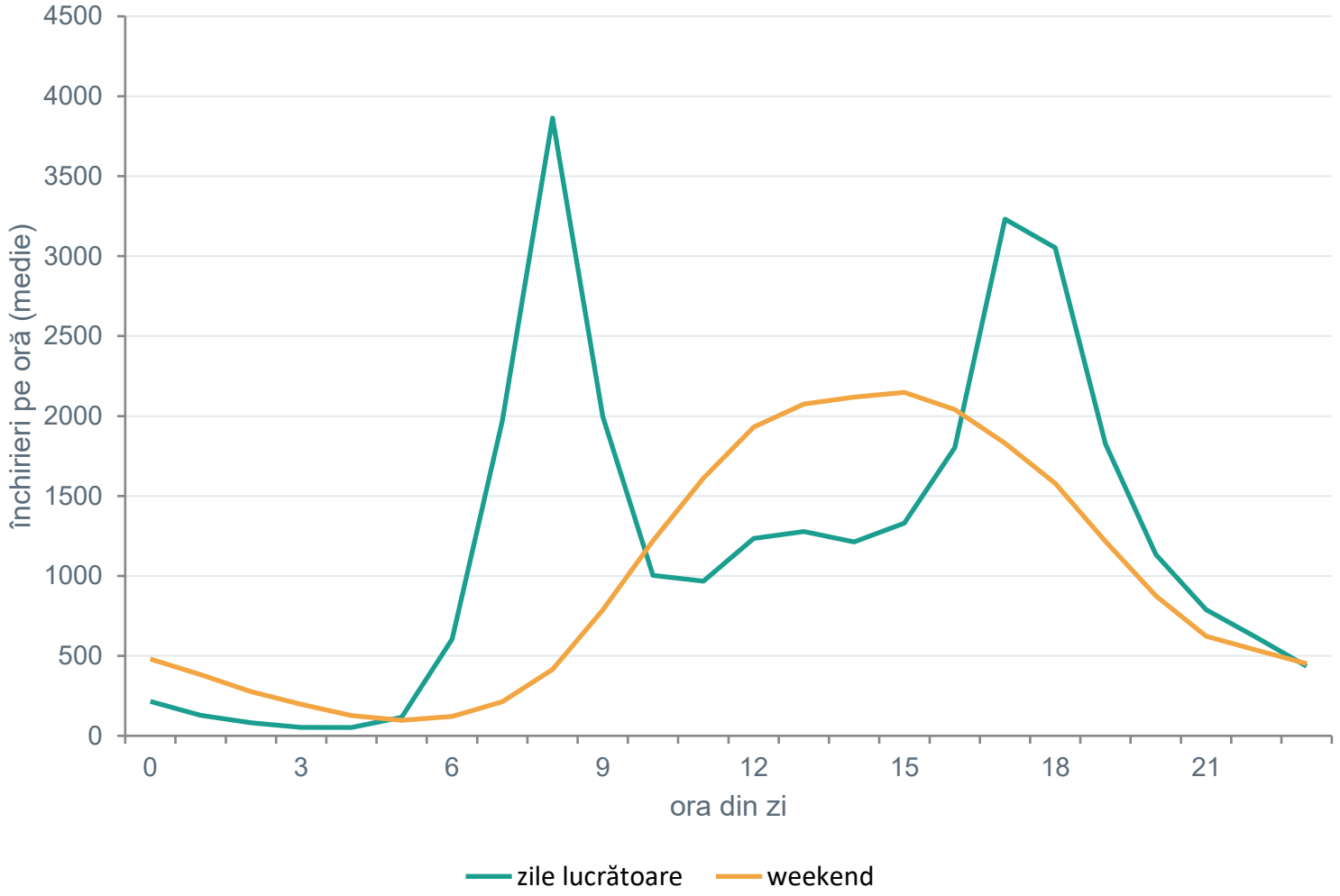
1. Setul de date: 2 ani, din oră în oră
2. Regresie liniară vs gradient boosting
3. Antrenare pe 2015, test pe 2016
4. Ce prinde și ce scapă fiecare model

Setul de date London bike sharing

Element	Descriere
Perioada	4 ianuarie 2015 – 3 ianuarie 2017, din oră în oră: **17 414** de înregistrări
Ținta (cnt)	numărul de biciclete închiriate în ora respectivă
Vremea	temperatura t1 și cea resimțită t2 (°C), umiditatea, viteza vântului, codul vremii (senin, ploaie, zăpadă…)
Calendarul	ora, zi de weekend, sărbătoare legală, sezonul
Împărțirea	antrenare: anul 2015 (8 643 de ore); test: anul 2016 (8 771 de ore)

- Sursa: Transport for London și freemeteo, publicat pe Kaggle; același fișier ca la laborator (london_merged.csv)
- Împărțirea **în timp** (trecut / viitor) imită folosirea reală: prezicem cererea de mâine din datele de ieri

Profilul orar: zilele lucrătoare și weekendul



- Zile lucrătoare: două vârfuri, la 8 (**3 864**) și la 17 (**3 232**) – naveta
- Weekend: o singură cocoașă, maximul la prânz (**2 148**)
- Relația cu ora este puternic **neliniară** și depinde de tipul zilei
- Corelația cu temperatura este doar **0,39**

Trei modele pentru aceeași țintă (scikit-learn)

```
tr = d[d.timestamp < "2016-01-01"]   # antrenare: 2015
te = d[d.timestamp >= "2016-01-01"]  # test: 2016

# 1) dreaptă pe temperatură
m1 = LinearRegression().fit(tr[["t1"]], tr.cnt)
# 2) liniar cu indicatori pentru oră + vreme
m2 = LinearRegression().fit(feats(tr), tr.cnt)
# 3) gradient boosting pe variabilele brute
cols = ["hour", "t1", "hum", "wind_speed",
        "is_weekend", "is_holiday", "weather_code"]
m3 = HistGradientBoostingRegressor(max_iter=300)
m3.fit(tr[cols], tr.cnt)
print(m3.score(te[cols], te.cnt))   # R² pe 2016
```

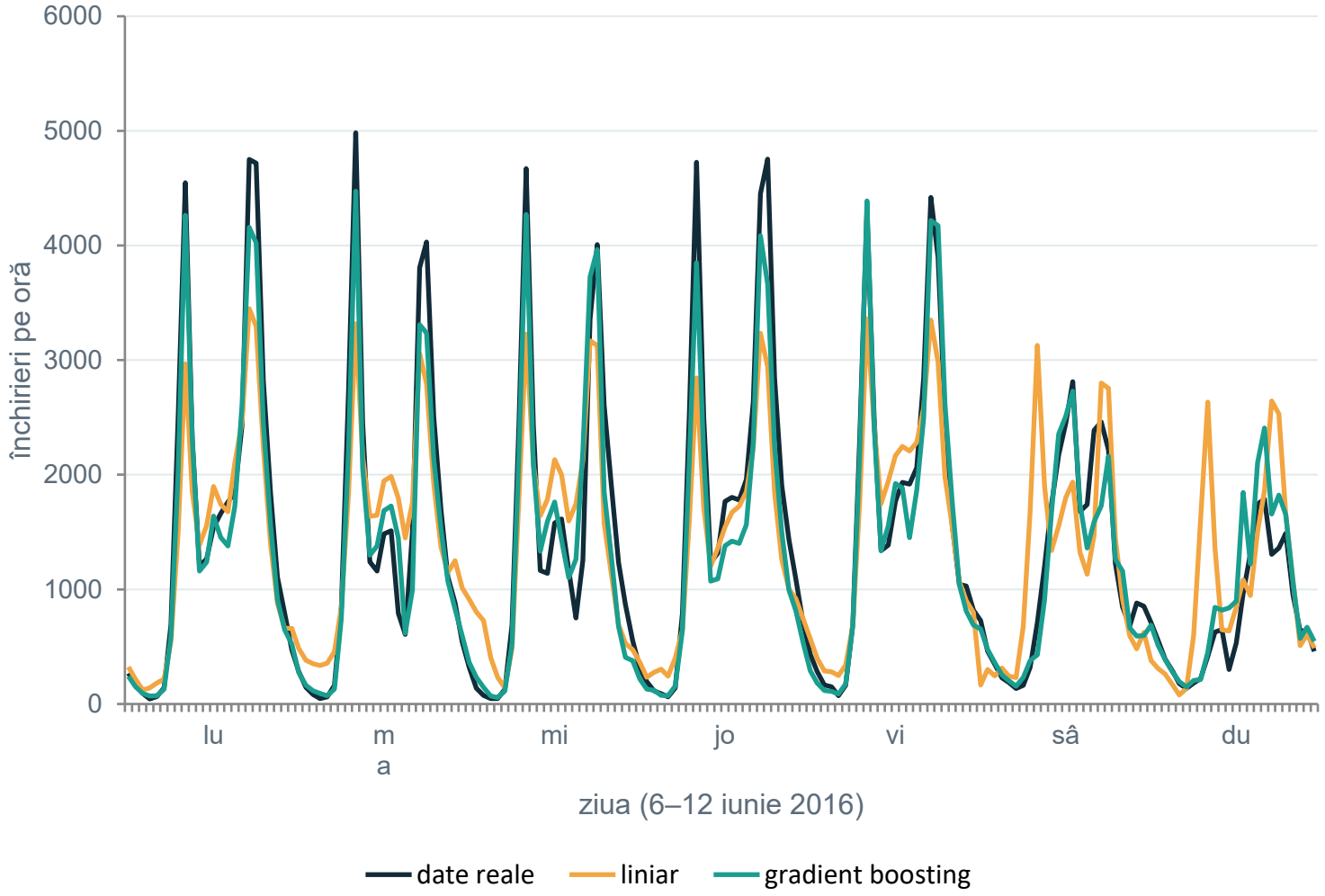
- feats: 24 de indicatori pentru oră + temperatură, umiditate, vânt, weekend, ploaie
- **Gradient boosting**: sute de arbori mici, fiecare corectează erorile celor de dinainte
- Antrenarea pe 8 643 de ore: **0,8 s**
- score întoarce R^2 – calculat pe anul **2016**

Rezultatele: ce câștigăm cu un model mai flexibil

Model	R^2 antrenare (2015)	R^2 test (2016)	Eroarea tipică (RMSE)
Dreaptă: cnt = 128 + 79·t1	0,15	0,15	–
Liniar cu ora și vremea	0,715	0,713	593 biciclete/oră
Gradient boosting	0,970	**0,934**	**284** biciclete/oră

- Temperatura singură explică doar **15%** din variație – ora contează mult mai mult
- Indicatorii pentru oră urcă R^2 la 0,71, dar profilul este același în toate zilele; arborii învață profiluri diferite pentru weekend
- Gradient boosting: eroarea scade la **jumătate**; diferența dintre antrenare și test (0,97 față de 0,93) este mică

O săptămână din iunie 2016, nevăzută la antrenare



- Liniar: vârful prezis cel mult **3 449**, față de 4 984 în realitate
- Gradient boosting urmărește vârfurile de navetă și forma de weekend
- Ambele greșesc în zilele atipice: ploaie bruscă, evenimente, greve
- Ce nu e în date (un eveniment în oraș) nu poate fi învățat

Verificare

? Gradient boosting are $R^2 = 0,97$ pe 2015 și 0,93 pe 2016. Ce valoare raportăm ca performanță a modelului?

A 0,97 – modelul a fost construit cu aceste date

B 0,93 – date pe care modelul nu le-a văzut

C media celor două: 0,95

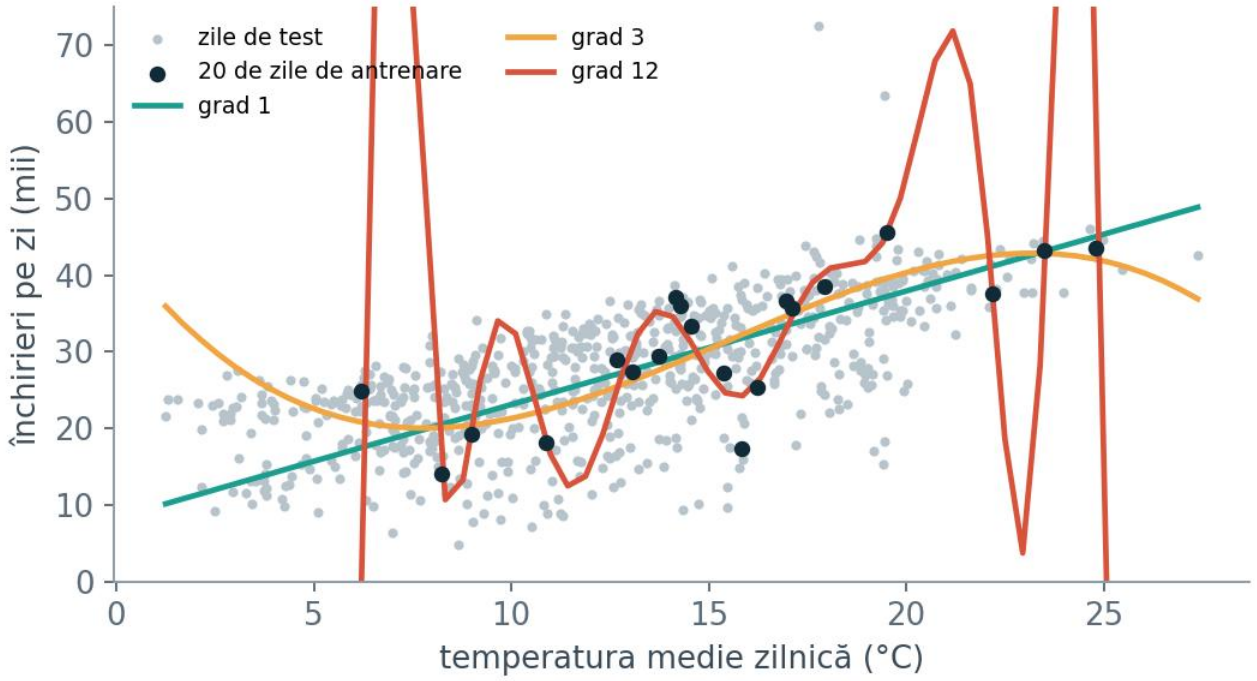
D oricare, diferența este neglijabilă

Capcana principală

Supraînvățare și generalizare

1. Polinoame de grad tot mai mare
2. Eroarea de antrenare vs eroarea de test
3. Validarea încrucișată
4. Legătura cu validarea modelelor de simulare (S6)

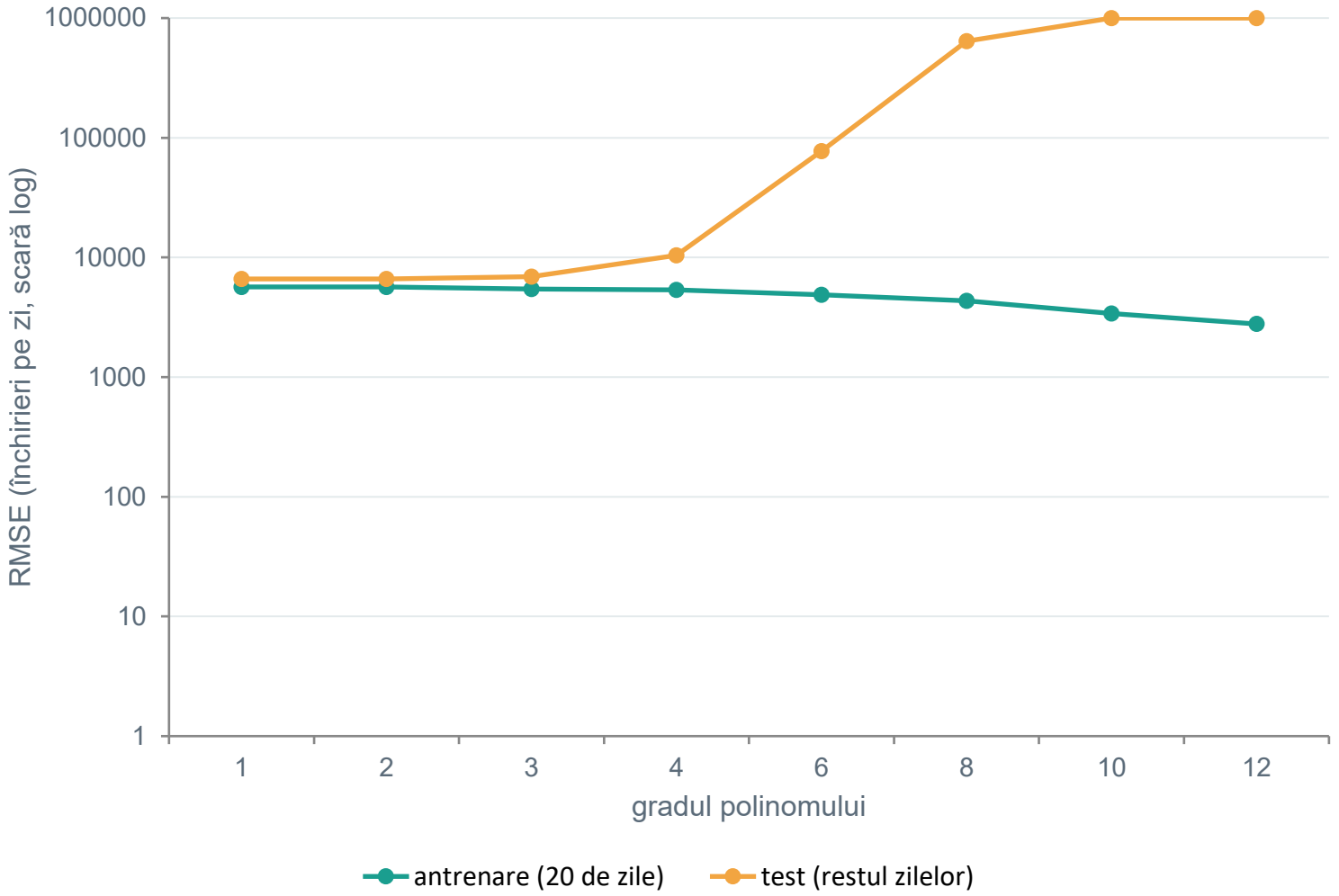
Același set de 20 de zile, trei polinoame



- **Grad 1**: prea simplu, dar stabil
- **Grad 3**: urmează curbura datelor
- **Grad 12**: trece aproape de toate punctele negre, dar oscilează sălbatic între ele
- Zilele de test (gri) arată adevărul: gradul 12 prezice valori absurde

închirieri pe zi în funcție de temperatura medie; 20 de zile de antrenare, restul de test

Eroarea de antrenare scade mereu; eroarea de test, nu



- Antrenare: de la 5 671 la **2 779** – scade mereu
- Test: minim la gradele 1–2 (~6 612), apoi explodează
- Gradul 8: eroare de test **642 869**; de la gradul 10, peste un milion
- Mai mulți parametri decât poate susține setul de date = supraînvățare

Cum evaluăm corect un model din date

- Trei seturi: **antrenare** (coeficienții), **validare** (alegerea modelului și a gradului), **test** (raportul final, folosit o singură dată)
- **Validare încrucișată** cu k părți: antrenăm de k ori, de fiecare dată fără o parte, și facem media erorilor
- Pentru serii de timp: împărțire **cronologică** (antrenăm pe trecut, testăm pe viitor), nu aleatoare
- **Regularizarea** penalizează coeficienții mari (ridge, lasso) și reduce supraînvățarea
- Scurgerea de informație: orice prelucrare (normalizare, selecție) se învață doar pe datele de antrenare

6 **Legătura cu S6**

Calibrarea unei simulări este antrenarea, iar validarea pe date nefolosite este testul. Aceeași disciplină se aplică ambelor tipuri de modele.

Fluxul de lucru pentru un model din date

1

Întrebarea

Ce prezicem, pentru cine, cu ce precizie e util?

2

Datele

Colectare, curățare, variabile explicative; împărțire înainte de orice

3

Model de bază

Cel mai simplu model: media, o dreaptă – reperul de bătut

4

Modele flexibile

Arbori, rețele; alegerea prin validare

5

Testul final

O singură evaluare pe datele de test

6

Monitorizarea

Lumea se schimbă: performanța scade în timp (deriva datelor)

Pasul uitat cel mai des este modelul de bază: fără el nu știm dacă un R^2 de 0,7 este bun sau slab. Iar monitorizarea leagă modelul de restul vieții sistemului – idee pe care o reluăm la gemenii digitali.

Demo-ul săptămânii

Modele surogat pentru simulări scumpe

1. Ideea: învățăm o simulare, nu un sistem real
2. Planul de experimente: Latin Hypercube
3. Surogatul unei cozi M/G/1: patru modele
4. Emulatorul epidemiei din S9 și calibrarea instantanee

Ce este un model surogat

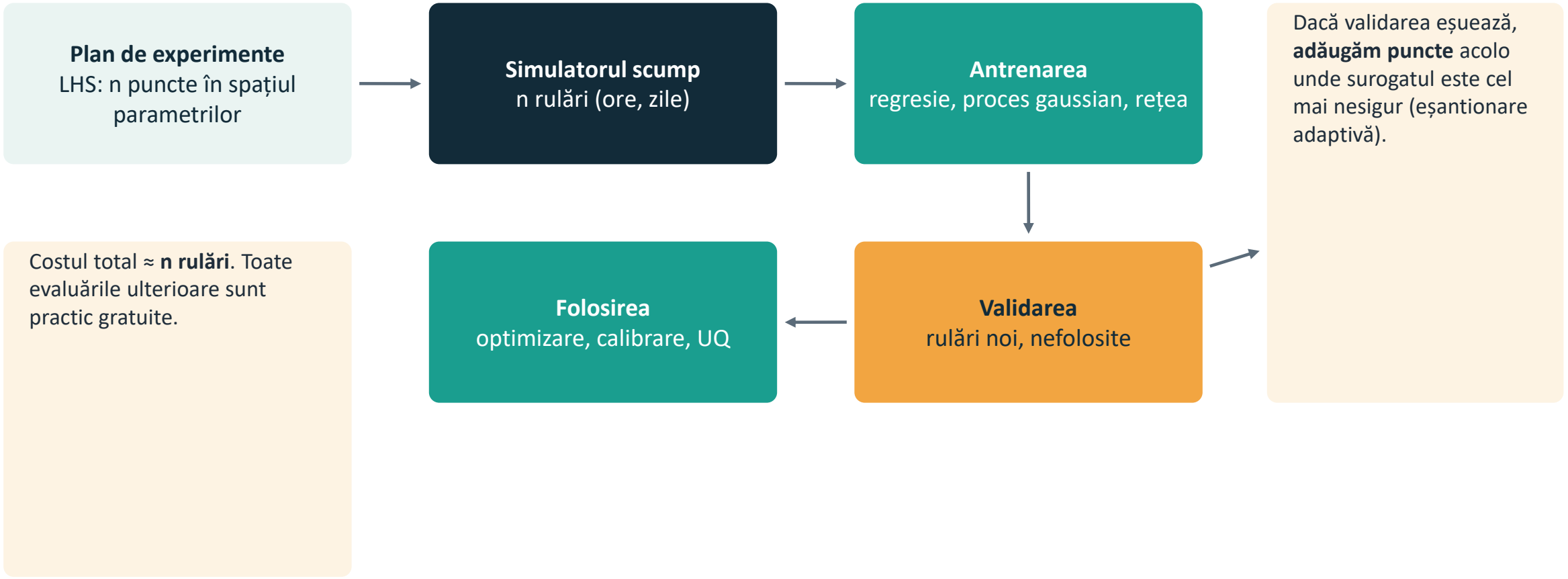
Un model rapid, învățat din rulări ale unei simulări lente

Un **surogat** (sau **emulator**, sau **metamodel**) aproximează funcția parametri ↦ rezultat a unui simulator. Se antrenează pe un număr mic de rulări alese cu grijă, apoi înlocuiește simulatorul acolo unde avem nevoie de mii sau milioane de evaluări.

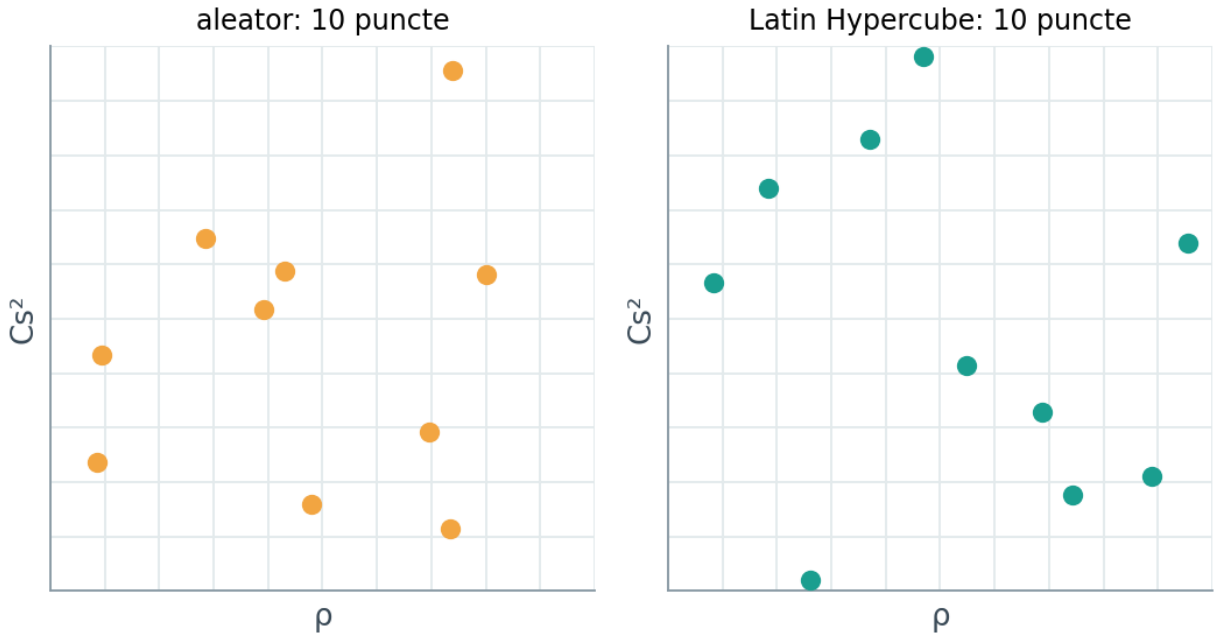
Kleijnen, Design and Analysis of Simulation Experiments, 2015; Kennedy și O'Hagan, 2001

- **Optimizare**: căutăm parametrii cei mai buni fără să rulăm simulatorul la fiecare pas
- **Calibrare**: ABC sau metode bayesiene cu sute de mii de evaluări
- **Analiza sensibilității și a incertitudinii** (S6): Monte Carlo pe parametri
- **Explorare interactivă**: răspuns instantaneu pentru decidenți

Construirea și folosirea unui surogat



Unde rulăm simulatorul: eșantionarea Latin Hypercube



- Aleator: goluri și aglomerări – rulări irosite
- **LHS** (McKay, 1979): împărțim fiecare parametru în n intervale și folosim fiecare interval **o singură dată**
- Acoperire uniformă pe fiecare axă, pentru orice număr de dimensiuni
- Grila completă crește exponențial: 10 valori pe 5 parametri = 100 000 de rulări

același 10 rulări: la LHS fiecare rând și fiecare coloană conține exact un punct

Demo: surogatul unei cozi M/G/1

```
def sim_queue(rho, cs2, n=20000, seed=0):  # Lindley (S4)
    g = np.random.default_rng(seed)
    A = g.exponential(1 / rho, n)            # sosiri
    S = g.gamma(1 / cs2, cs2, n)             # servire, medie 1
    w = tot = 0.0
    for k in range(n):
        tot += w + S[k]; w = max(0.0, w + S[k] - A[k])
    return tot / n                           # W mediu

X = lhs(80)                    # ρ ∈ [0,1; 0,9], Cs² ∈ [0,2; 3]
y = [sim_queue(*x, seed=i) for i, x in enumerate(X)]
gp = GaussianProcessRegressor(C() * RBF([.3, 1]) + White())
gp.fit(X, np.log(y))           # învățăm log W
mu, sd = gp.predict(X_nou, return_std=True)
```

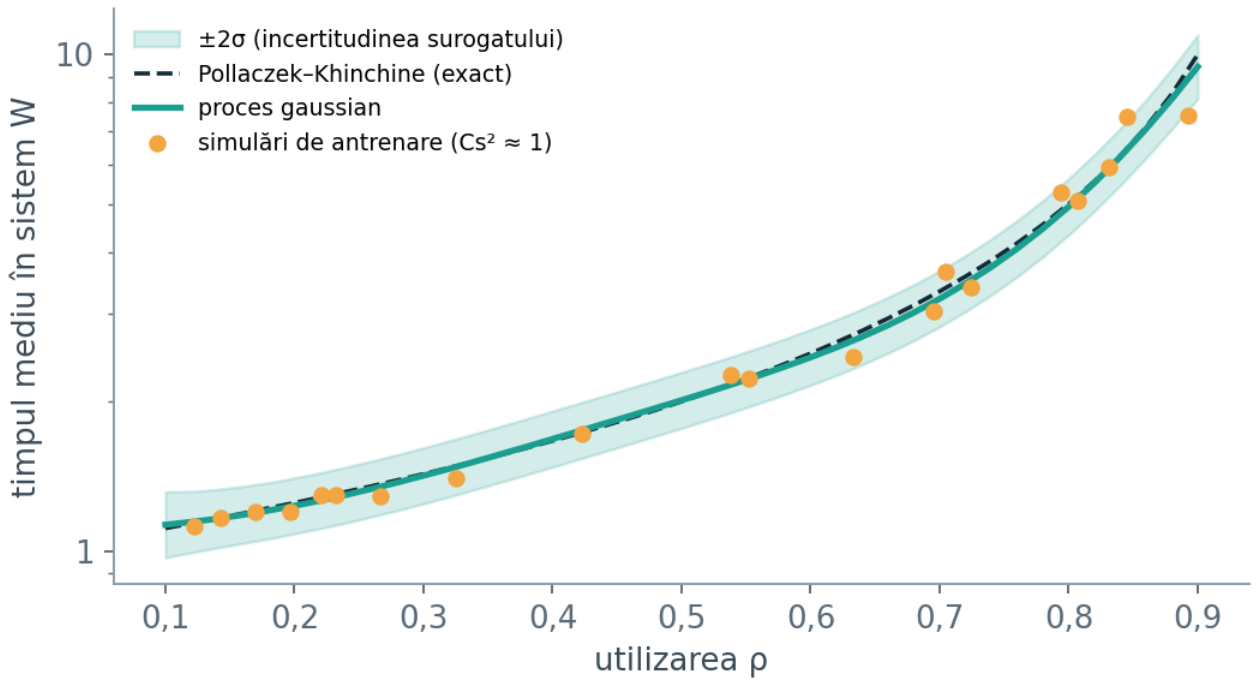
- O simulare: 20 000 de clienți, **15,3 ms** în bucla Python
- Doi parametri: utilizarea ρ și variabilitatea servirii Cs^2
- Răspunsul exact este cunoscut (Pollaczek–Khinchine, S4) – putem măsura eroarea
- Testăm pe 300 de puncte noi, alese aleator

Patru surogate, antrenate pe aceleași 80 de simulări

Surogat	Eroare medie	Eroare maximă	Timp / predicție
Regresie liniară	13,2%	41,7%	0,8 µs
Polinom de grad 3	2,2%	9,3%	1,6 µs
Rețea neurală (2 × 32 de neuroni)	5,5%	26,7%	1,1 µs
Proces gaussian	**1,6%**	11,6%	1,6 µs
O singură simulare (referință)	3,4%	–	15,3 ms

- Erorile sunt relative, față de formula exactă, pe 300 de puncte de test; timpul surogatului este măsurat pe lot
- Procesul gaussian și polinomul sunt **mai precise decât o simulare** – netezesc zgomotul celor 80 de rulări
- Rețeaua neurală are nevoie de mai multe date: cu 80 de puncte, modelele simple câștigă

Procesul gaussian știe și cât de sigur este



secțiune la $Cs^2 = 1$ (coada M/M/1); scară logaritmică pe verticală

- Linia teal: predicția; banda: **±2σ**
- Banda se lărgește acolo unde **nu avem rulări** și unde simularea e zgomotoasă (ρ mare)
- Incertitudinea spune **unde** să adăugăm următoarele rulări
- Kriging: aceeași metodă, inventată în geostatistică (Krige, Matheron, anii ’60)

Cât câștigăm cu surogatul

9 569×

mai rapid decât o simulare: microsecunde în loc de milisecunde

1,6%

eroare medie a surogatului, față de **3,4%** pentru o simulare

80

simulări **folosite la antrenare** – costul total al surogatului

- Pentru o simulare de o oră, același raport transformă o săptămână de calcul în câteva secunde
- Câștigul contează doar dacă avem nevoie de **multe** evaluări: optimizare, calibrare, analiza incertitudinii

Lecția scării: de ce am învățat log W

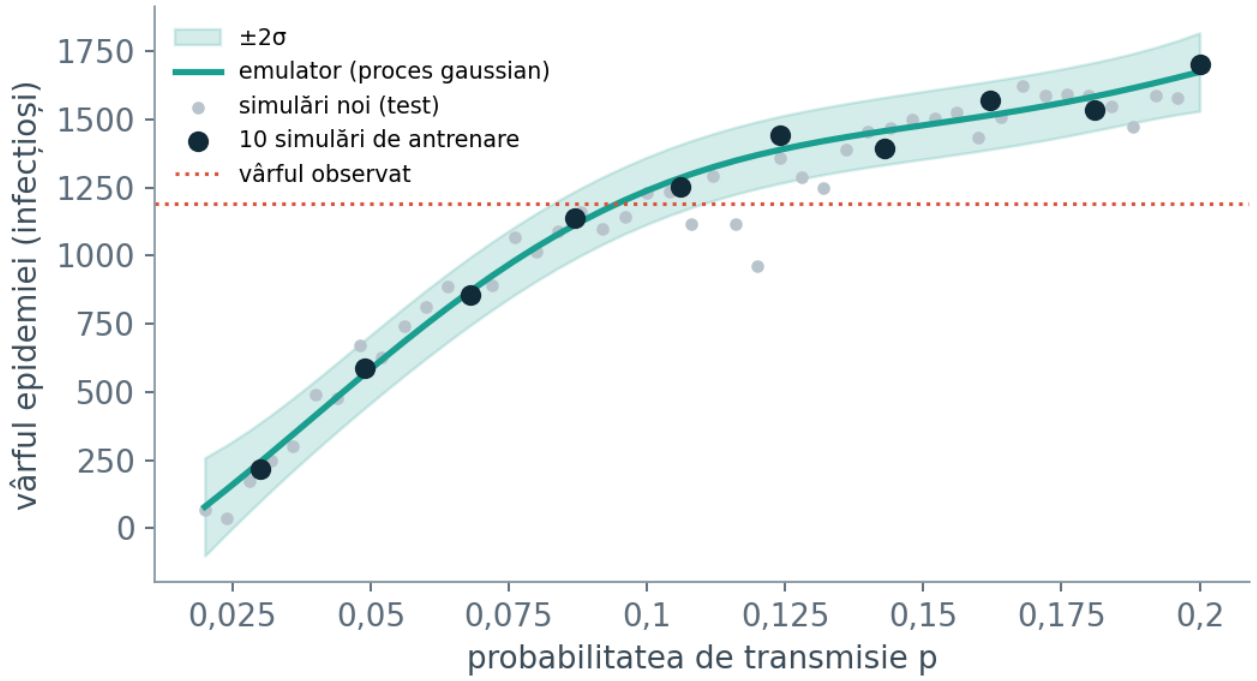
- W crește exploziv când ρ se apropie de 1: de la 1,1 la ρ = 0,1 până la peste 10 la ρ = 0,9
- Polinom de grad 3 pe W direct: eroare medie **14,2%**, maximă **110,7%**
- Același polinom pe log W: eroare medie **2,2%** – de 6 ori mai bine, cu aceleași date
- Pe scara log, erorile **relative** au aceeași greutate peste tot; pe scara directă domină punctele cu W mare
- Cunoașterea sistemului ajută: o variabilă 1/(1 − ρ), inspirată din formula P-K, ar face problema aproape liniară

G

Cutie gri, din nou

Cea mai bună îmbunătățire nu a venit dintr-un model mai complicat, ci din înțelegerea sistemului: știm din S4 că timpul de așteptare crește ca 1/(1 − ρ).

Emulatorul epidemiei din S9: 10 simulări de antrenare



- O simulare: 0,17 s; aici am putea rula oricât, dar ne prefacem că e scumpă
- Vârful epidemiei în funcție de probabilitatea de transmisie p
- **89%** dintre cele 46 de simulări noi cad în banda ±2σ (ideal: ~95%)
- Zgomotul dintre replicări este învățat de termenul WhiteKernel

modelul Holban redus: 2 500 de agenți, 5 × 5 km, 120 de zile

Calibrare ABC cu emulatorul, în loc de simulator

0,05 s

100 000 de evaluări ale emulatorului

4,8 ore

cât ar fi durat aceleași evaluări cu **simulatorul**

0,098 ± 0,012

p estimat din vârful observat (1 190); **adevărat: 0,1**

- Ponderăm fiecare eșantion a priori după cât de aproape prezice emulatorul vârful observat, ținând cont de zgomotul replicărilor (66, din S9) și de incertitudinea emulatorului
- Răspunsul la întrebarea din S9: pentru o simulare de o oră, 10 rulări (10 ore) înlocuiesc 100 000 de ore de calcul

Surogate și emulatoare în practică

1 Proiectare în inginerie

Aripi, motoare, caroserii: mii de variante evaluate pe surogat, doar cele bune trec prin simularea CFD completă.

2 Teste de impact

O simulare de accident durează ore; surogatele permit optimizarea grosimii tablelor și a structurii.

3 Climă

Emulatoare ale modelelor climatice explorează scenarii de emisii fără a rula din nou modelul complet.

4 Proiectarea cipurilor

Surogate pentru timing și consum în spațiul parametrilor de proiectare.

5 Hiperparametri AI

Optimizarea bayesiană: un proces gaussian alege următoarea configurație de antrenat.

6 Epidemiologie

Emulatoare ale modelelor cu agenți pentru calibrare și analiza politicilor (ca în demo).

Același tipar peste tot: simulatorul este prea scump pentru întrebarea pusă, dar suficient de ieftin pentru câteva zeci sau sute de rulări bine alese.

Verificare

? Avem un buget de **50 de rulări** ale unei simulări de 2 ore, cu 3 parametri. Ce surogat și ce plan alegem?

A rețea neurală adâncă, puncte pe o grilă completă

B proces gaussian, puncte Latin Hypercube

C regresie liniară, toate punctele în centrul domeniului

D nu folosim surogat: rulăm direct 100 000 de simulări

Ora 2

Fizică + învățare automată; simulare pentru AI

1. Prognoza meteo cu AI: cel mai mare surogat din lume
2. Rețele informate de fizică (PINN) și operatori neurali
3. Descoperirea ecuațiilor din date (SINDy)
4. Simularea care antrenează AI: sim-to-real și randomizarea domeniului

Ce aduce fizica și ce aduc datele

Fizica (ecuațiile)

- **conservare**: masă, energie, impuls
- extrapolează în condiții nevăzute
- puțini parametri, cu sens
- dar: procese necunoscute sau prea fine
- și: simulare lentă la rezoluție mare

vs

Datele (învățarea)

- captează ce nu știm să scriem
- **rapid** după antrenare
- se îmbunătățește cu mai multe date
- dar: nu respectă automat legile fizicii
- și: slab în afara datelor de antrenare

Modelarea științifică bazată pe învățare (scientific machine learning) caută combinații în care fiecare parte acoperă slăbiciunile celeilalte: fizica dă structura și garanțiile, datele dau detaliile și viteza.

Cinci moduri de a combina fizica și învățarea

Abordarea	Ideea	Exemplu
Surogat / emulator	rețeaua învață ieșirile unui simulator din rulări ale lui	GraphCast, GenCast; demo-ul M/G/1
Model hibrid	ecuațiile rămân, iar rețeaua învață doar termenul necunoscut	NeuralGCM: dinamică + fizică învățată
PINN	ecuația intră în funcția de cost a rețelei	răcirea cafelei cu 6 măsurători
Operator neural	învață soluția unei clase întregi de ecuații	FNO pentru curgeri, prognoze
Descoperirea ecuațiilor	găsește formula însăși, dintr-o bibliotecă de termeni	SINDy pe Lotka–Volterra

- De sus în jos crește cantitatea de fizică păstrată în model
- Simularea diferențiabilă (gradient prin simulator) face posibile ultimele patru abordări

Studiu de caz

Prognoza meteo: cel mai mare surogat din lume

1. Prognoza numerică clasică
2. GraphCast, GenCast, NeuralGCM, AIFS
3. Ce câștigăm și ce pierdem

Prognoza numerică clasică (NWP)

- Ecuațiile atmosferei: Navier–Stokes pe sferă în rotație, termodinamică, vapori de apă – rezolvate pe o grilă globală
- ECMWF (Centrul European): grilă de ~**9 km**, peste 100 de niveluri verticale, pași de câteva minute
- **Asimilarea datelor**: milioane de observații pe zi (sateliți, baloane, avioane, stații) fixează starea inițială
- Ansambluri de ~50 de rulări cu stări inițiale perturbate dau **incertitudinea** (haos, S7)
- **Reanaliza ERA5**: starea atmosferei reconstruită oră de oră din 1940 până azi – setul de antrenare pentru AI

R **Richardson, 1922**

Prima prognoză numerică a fost calculată de mână și a durat săptămâni. Richardson imagina o „fabrică de prognoze” cu 64 000 de calculatoare umane într-o sală sferică.

Modelele meteo învățate

1 GraphCast (2023)

Google DeepMind, Science: rețea pe graf, 0,25°, 10 zile în sub un minut pe un TPU; mai bun decât HRES pe ~90% din ținte.

2 GenCast (2024)

Nature, decembrie: model de difuzie care generează un ansamblu de 50+ prognoze pe 15 zile.

3 NeuralGCM (2024)

Nature: model hibrid – nucleu dinamic clasic, diferențiabil, plus fizică învățată; stabil și pentru simulări climatice.

4 AIFS Single (2025)

ECMWF: primul model AI operațional al Centrului European, din februarie 2025.

5 AIFS ENS (2025)

ECMWF, operațional din 1 iulie 2025: ansamblu AI, câștig de până la 20% față de ansamblul fizic.

6 Aurora (2025)

Microsoft, Nature: model de bază pentru sistemul Pământului – vreme, calitatea aerului, valuri, cicloni.

Toate sunt antrenate pe decenii de reanaliză ERA5. Majoritatea au codul și ponderile publice – le puteți rula pe un singur GPU.

GenCast față de cel mai bun ansamblu fizic (ENS)

97,2%

dintre cele 1 320 de ținte evaluate: GenCast este **mai precis** (99,8% peste 36 de ore)

8 min

o prognoză pe **15 zile**, pe un singur TPU v5

0,25°

rezoluție (~28 km), antrenat pe **40 de ani** de ERA5

- Ansamblul fizic ENS rulează ore întregi pe un supercalculator cu zeci de mii de nuclee
- AIFS ENS (ECMWF, 2025): de peste 10 ori mai rapid și cu ~1 000 de ori mai puțină energie decât ansamblul fizic

Ce pierdem: limitele modelelor meteo învățate

- Învață din ERA5, care este produsă de **modele fizice** și asimilare – depind de fizică pentru date
- Starea inițială a fiecărei prognoze vine tot din **asimilarea** clasică a observațiilor
- Evenimentele **extreme** rare (record de temperatură, uragane neobișnuite) sunt slab reprezentate în date
- Rezoluție mai mică: AIFS la ~31 km față de 9 km pentru modelul fizic; câmpurile pot fi prea netede
- Nu garantează conservarea și coerența fizică; schimbarea climei mută datele în afara zonei de antrenare

L **Lecția**

Modelele fizice nu dispar: produc datele de antrenare, starea inițială și referința pentru extreme. Centrul European rulează ambele tipuri în paralel.

Verificare

? De ce modelele meteo AI **nu pot înlocui complet** astăzi modelele fizice?

A sunt mai lente decât modelele fizice

B datele de antrenare și starea inițială provin din modele fizice cu asimilare

C nu pot produce ansambluri

D nu au fost niciodată folosite operațional

Fizica în funcția de cost

Rețele neurale informate de fizică (PINN)

1. Ideea: ecuația devine un termen de penalizare
2. Demo: răcirea cafelei din 6 măsurători
3. Estimarea parametrului k în același timp
4. Operatori neurali și simulare diferențiabilă

Ce este o PINN

O rețea neurală antrenată pe date și pe ecuație în același timp

Raissi, Perdikaris și Karniadakis (2019): o rețea $T(t; \theta)$ aproximează soluția. Funcția de cost conține eroarea față de măsurători **plus** reziduul ecuației diferențiale în multe **puncte de colocare**, unde nu avem date.

M. Raissi, P. Perdikaris, G. E. Karniadakis, Journal of Computational Physics 378, 2019

- Derivata dT/dt se calculează exact prin **diferențiere automată** (aici: de mână, rețeaua e mică)
- Parametrii necunoscuți ai ecuației (k) se învață **împreună** cu ponderile rețelei – problemă inversă
- Nu cere o grilă: punctele de colocare pot fi oriunde, în orice dimensiune
- Ecuația acționează ca o **regularizare**: rețeaua nu poate oscila liber între măsurători

Funcția de cost a PINN pentru răcirea lui Newton

$$J(\theta, k) = (1/N)\cdot\Sigma\,(T(t_i) - T_i)^2 \;+\; \lambda\cdot(1/M)\cdot\Sigma\,(dT/dt(\tau_j) + k\cdot(T(\tau_j) - 20))^2$$

$T(t) = T(t; \theta)$ rețeaua: 12 neuroni tanh, **38 de parametri** (inclusiv log k)

(t_i, T_i), $N = 6$ măsurătorile, toate în **primele 8 minute**, cu zgomot de ~0,8 °C

τ_j, $M = 41$ punctele de colocare pe [0; 40] minute – aici impunem doar **ecuația**

$\lambda = 100$ ponderea fizicii; prea mică – rețeaua ignoră ecuația, prea mare – ignoră datele

Ecuația din S7: $dT/dt = -k\cdot(T - T_a)$, cu $T_a = 20$ °C. Valoarea adevărată k = 0,08 pe minut nu este dată rețelei: k pornește de la 0,03 și este învățat. Optimizare cu L-BFGS-B din SciPy, 6 porniri aleatoare.

PINN fără biblioteci de deep learning (NumPy + SciPy)

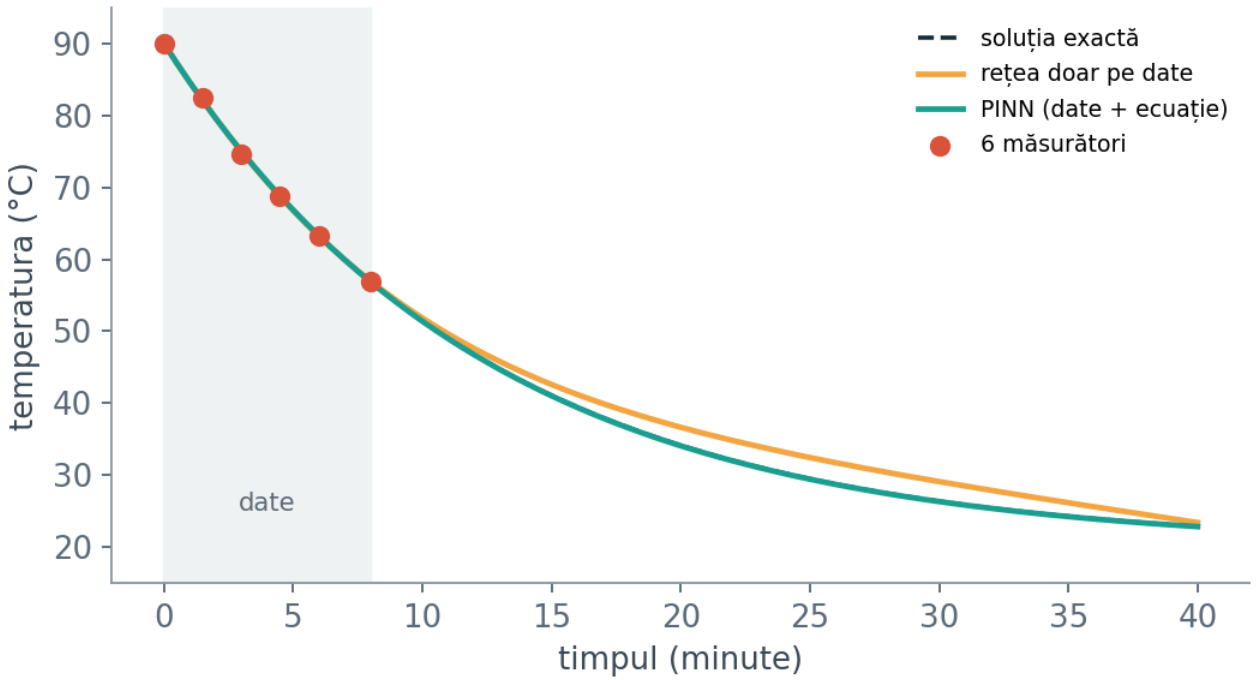
```
def net(th, t):                  # 1 strat ascuns, 12 neuroni
    W1, b1, W2, b2 = th[:12], th[12:24], th[24:36], th[36]
    z = np.tanh(np.outer(t / 40, W1) + b1)
    T = 20 + 70 * (z @ W2 + b2)
    dT = 70 * ((1 - z**2) * W1 / 40) @ W2   # dT/dt exact
    return T, dT

def cost(th):
    T, _ = net(th, t_obs)
    k = np.exp(th[-1])                      # k > 0
    Tc, dTc = net(th, t_col)                # colocare
    return (np.mean((T - T_obs)**2)
            + 100 * np.mean((dTc + k * (Tc - 20))**2))

res = minimize(cost, th0, method="L-BFGS-B")
```

- Derivata rețelei se scrie analitic: $(\tanh)' = 1 - \tanh^2$
- $k = \exp(\theta)$ garantează $k > 0$
- Varianta „doar date”: același cod, fără termenul de fizică
- Cu PyTorch sau JAX, derivata vine automat

Aceleași 6 măsurători, cu și fără fizică



zona gri: intervalul cu date; după minutul 8 rețelele extrapolează

- În zona cu date, ambele rețele trec prin puncte
- Doar date: după minutul 8 deviază, eroare maximă **3,0 °C**
- PINN: urmează soluția exactă până la minutul 40, eroare maximă **0,07 °C**
- Ecuația „completează” intervalul fără măsurători

Rezultatele PINN

0,0803

constanta de răcire **învățată** (adevărat: 0,08)

1,89 / 0,05

RMSE pe 0–40 min (°C): **doar date** / **PINN**

38 / 6

parametri / măsurători: fără fizică, problema ar fi **subdeterminată**

- Problema inversă (aflarea lui k) și cea directă (curba completă) se rezolvă într-un singur pas
- Pentru această ecuație simplă, o regresie neliniară pe soluția exactă (S7) ar fi suficientă; PINN contează când soluția nu are formulă

Când ajută PINN și când nu

- **Ajută**: probleme inverse (parametri necunoscuți), date rare și zgomotoase, geometrii complicate, dimensiuni mari
- **Ajută**: combinarea măsurătorilor cu ecuații incomplete – de exemplu fluxul sanguin din imagini medicale
- **Nu ajută**: problemele directe bine puse – un solver clasic (RK45, elemente finite) este mai rapid și mai precis
- Antrenarea este dificilă: ponderea λ, sisteme rigide, soluții cu fronturi abrupte sau oscilații rapide
- Nu există garanții de eroare ca la metodele numerice din S7 – rezultatul trebuie verificat

! Atenție la promisiuni

Pentru simularea directă a unei ecuații cunoscute, metodele numerice clasice rămân de obicei mai bune. Avantajul real apare când combinăm date incomplete cu fizică incompletă.

Operatori neurali: învățăm toate soluțiile odată

- O PINN rezolvă **o** problemă; un **operator neural** învață aplicația condiție inițială ↦ soluție pentru o clasă întreagă
- **FNO** (Li et al., ICLR 2021): straturi care lucrează în spațiul Fourier; **DeepONet** (Lu et al., 2021)
- Antrenat pe mii de rulări ale unui solver clasic, apoi rezolvă probleme noi în milisecunde
- Independent de rezoluție: antrenat pe o grilă grosieră, evaluat pe una fină
- Aplicații: curgeri turbulente, stocarea CO_2 în subteran, prognoză (FourCastNet folosește o variantă FNO)

S **Legătura cu surogatele**

Un operator neural este un surogat al cărui intrare și ieșire sunt **funcții** (câmpuri), nu câteva numere ca în demo-ul M/G/1.

Simularea diferențiabilă: gradientul prin simulator

- Simulatorul este scris într-un cadru cu **diferențiere automată**: JAX, PyTorch, NVIDIA Warp, MuJoCo MJX
- Putem calcula ∂(rezultat)/∂(parametri) prin toți pașii simulării, la costul a încă o simulare
- **Calibrare** prin coborâre pe gradient, în locul a sute de rulări (ABC, S9)
- **Control și proiectare**: optimizăm traiectoria unui robot sau forma unei piese direct prin simulator
- NeuralGCM se antrenează **prin** nucleul dinamic diferențiabil: rețeaua învață ce lipsește ecuațiilor

L **Limita**

Gradientul există doar dacă simularea este netedă: evenimentele discrete (S5), coliziunile și haosul (S7) fac gradientul inutil sau instabil.

Știință din date

Descoperirea ecuațiilor: SINDy

1. O bibliotecă de termeni candidați
2. Regresie rară: cât mai puțini termeni
3. Lotka–Volterra regăsit din date
4. Efectul zgomotului

SINDy: regresie rară pe o bibliotecă de termeni

$$dX/dt \approx \Theta(X)\cdot\Xi$$

X	datele: x(t) (prada) și y(t) (prădătorii), 600 de momente; derivatele se estimează numeric
$\Theta(X)$	biblioteca: coloanele **1, x, y, x^2, xy, y^2** evaluate în fiecare moment
Ξ	coeficienții, câte o coloană pentru fiecare ecuație; vrem cât mai mulți **zero**
STLSQ	cele mai mici pătrate, anulăm coeficienții sub prag (0,05), reajustăm restul; repetăm

Brunton, Proctor și Kutz, PNAS 2016. Ideea: majoritatea legilor fizicii au puțini termeni. Pragul este singurul parametru important – prea mic păstrează termeni falși, prea mare elimină termeni reali.

Lotka–Volterra regăsit din date

Date	ẋ: x	ẋ: xy	ẏ: y	ẏ: xy	Termeni ≠ 0 (din 12)
Modelul adevărat	1,1	−0,4	−0,4	0,1	4
Fără zgomot	1,099	−0,4	−0,4	0,1	4
Zgomot 1%	1,1	−0,4	−0,399	0,1	4
Zgomot 5% (netezit)	1,078	−0,388	−0,396	0,098	4

- Modelul: $\dot{x} = 1{,}1x - 0{,}4xy$, $\dot{y} = -0{,}4y + 0{,}1xy$; toți ceilalți 8 coeficienți ies **exact zero**
- Cu 5% zgomot, coeficienții deviază cu ~2–3%, dar **structura** ecuațiilor este regăsită corect
- Derivatele din date zgomotoase sunt partea grea: netezim cu un filtru Savitzky–Golay înainte de derivare

Ce poate și ce nu poate descoperirea ecuațiilor

- Rezultatul este o **formulă** citibilă, nu o cutie neagră – se poate interpreta, verifica, extrapola
- Biblioteca exprimă ce credem despre sistem: un termen care **lipsește** din bibliotecă nu poate fi găsit
- Coordonatele contează: aceeași dinamică poate fi rară într-un sistem de coordonate și densă în altul
- Variante: **regresia simbolică** (PySR, AI Feynman) caută și forma termenilor, prin căutare genetică
- Kepler a făcut același lucru de mână: legile mișcării planetelor din măsurătorile lui Tycho Brahe

M

Model din date, dar mecanist

SINDy închide cercul săptămânii: pornim de la date și ajungem la un model mecanist, cu parametri care au sens – cutie albă obținută din date.

Verificare

? Rulăm SINDy pe datele Lotka–Volterra, dar biblioteca are doar termenii **1, x, y, x^2, y^2** (fără xy). Ce obținem?

A exact modelul adevărat, cu alți coeficienți

B un model aproximativ, cu mai mulți termeni, care nu este cel adevărat

C SINDy adaugă automat termenul lipsă

D o eroare: algoritmul nu converge

Direcția inversă

Simularea care antrenează AI

1. De ce antrenăm în simulare
2. Bucla agent–simulator și uneltele
3. Decalajul sim-to-real – demo
4. Robotică, mașini autonome, riscuri

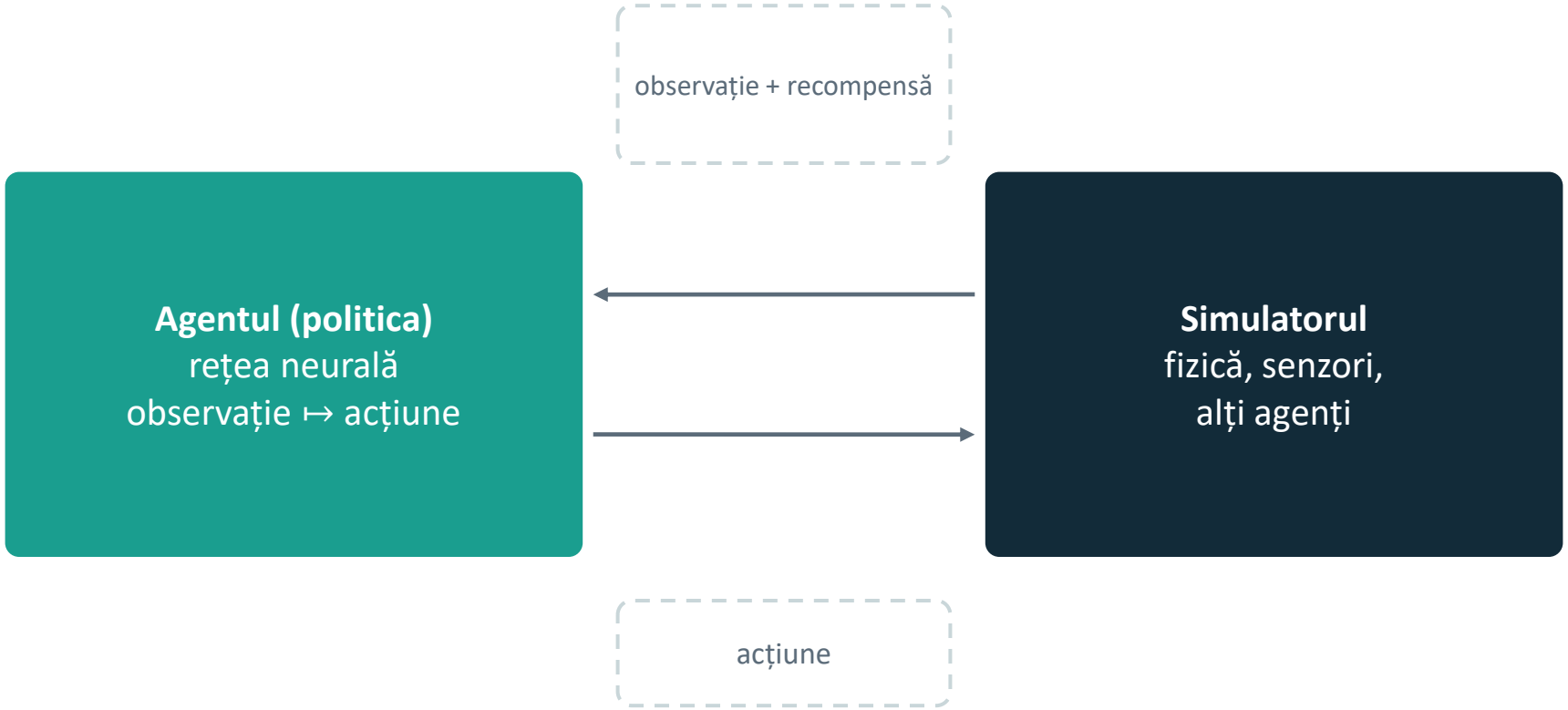
De ce antrenăm AI în simulare

- **Siguranță**: un robot care cade sau o mașină care lovește un pieton în simulare nu costă nimic
- **Viteză**: mai repede decât timpul real și mii de copii în paralel pe un GPU – ani de experiență într-o zi
- **Evenimente rare**: copilul care fuge în stradă, defecțiuni, furtuni – generate la cerere, cu variații
- **Etichete perfecte**: poziția exactă a fiecărui obiect, fără adnotare manuală
- **Reproductibilitate**: același scenariu, aceeași sămânță (S2), pentru a compara versiuni ale sistemului

Î **Învățarea prin întărire**

Agentul învață prin încercare și eroare, din recompense. Are nevoie de milioane de episoade – practic imposibil fără simulare.

Bucla de antrenare: agent și simulator



Milioane de episoade, mii de medii **în paralel** pe GPU.

Politica învățată este copiată pe robotul **real** – dacă simulatorul a fost suficient de fidel.

Simulatoare folosite pentru antrenarea AI

1 MuJoCo

Fizică pentru roboți cu contacte; Google DeepMind, open source din 2022; MJX rulează pe GPU.

2 Isaac Sim / Isaac Lab

NVIDIA: mii de roboți simulați în paralel pe un GPU, cu senzori și imagini realiste.

3 Gymnasium

Interfața standard reset / step pentru medii de învățare prin întărire (succesorul OpenAI Gym).

4 CARLA

Simulator open source pentru conducerea autonomă: orașe, trafic, vreme, senzori.

5 Motoare de jocuri

Unity, Unreal: medii vizuale bogate, date sintetice pentru recunoașterea imaginilor.

6 Modele ale lumii

Rețele generative care învață să simuleze din video (Genie 3, Cosmos): simulatorul este el însuși învățat.

Ultimul card închide cercul: simulatoarele pentru AI devin la rândul lor modele învățate din date – surogate ale lumii reale.

Decalajul sim-to-real

Diferența dintre lumea simulată și cea reală, pe care politica o „simte”

O politică optimizată în simulare exploatează **toate** detaliile simulatorului, inclusiv pe cele greșite. Pe sistemul real, unde aceste detalii diferă, performanța poate scădea dramatic.

Jakobi, Husbands, Harvey, 1995; Zhao et al., Sim-to-real transfer in deep RL for robotics: a survey, 2020

- **Parametri** greșiți: masă, frecare, rigiditate, coeficienți ai motorului
- **Dinamică nemodelată**: întârzieri, jocuri mecanice, saturări, uzură
- **Senzori**: zgomot, latență, calibrare; imagini simulate vs camere reale
- Optimizarea **amplifică** erorile modelului: caută exact colțurile în care simulatorul greșește

Cum reducem decalajul

- **Identificarea sistemului**: calibrăm simulatorul pe măsurători reale (S7, S9) – necesar, dar niciodată perfect
- **Randomizarea domeniului** (Tobin et al., 2017): antrenăm pe multe simulatoare cu parametri aleatori
- Ideea: dacă politica funcționează pe toată familia, realitatea arată ca „încă o variantă”
- **Randomizare automată**: intervalele se lărgesc pe măsură ce politica se descurcă (OpenAI, 2019)
- **Ajustare pe date reale**: câteva minute de experiență reală după antrenarea în simulare

U **Legătura cu S6**

Randomizarea domeniului este analiza incertitudinii folosită pentru proiectare: în loc să optimizăm pentru valoarea nominală, optimizăm pentru întreaga distribuție a parametrilor.

Demo: un controler PD reglat în simulare

	Simulatorul nominal	Sistemul „real”
Masa m	1,0	**1,6** (mai greu)
Frecarea b	0,5	**0,3** (mai mică)
Întârzierea actuatorului	0	**0,12 s** (6 pași)
Saturarea motorului	±20	±20
Randomizare (12 medii)	m ∈ [0,6; 2], b ∈ [0,2; 0,8], întârziere 0–0,16 s	–

- Sarcina: duce poziția x de la 0 la 1 și o ține acolo; costul = $\int (x-1)^2\,dt$ pe 10 secunde
- Controlerul: $u = K_p\cdot(1-x) - K_d\cdot v$; căutăm Kp și Kd pe o grilă de 50 × 21 de valori
- Sistemul real este **în interiorul** intervalelor de randomizare, dar nu a fost văzut la antrenare

Reglarea pe simulatorul nominal și cu randomizare

```
def cost(Kp, Kd, m=1.0, b=0.5, delay=0, dt=0.02):
    x = v = 0.0; buf = [0.0] * (delay + 1); J = 0.0
    for k in range(500):                    # 10 s
        u = np.clip(Kp*(1 - x) - Kd*v, -20, 20)
        buf.append(u); ua = buf.pop(0)      # întârziere
        v += dt * (ua / m - b * v); x += dt * v
        J += dt * (x - 1)**2
    return J

medii = [dict(m=g.uniform(.6, 2), b=g.uniform(.2, .8),
              delay=g.integers(0, 9)) for _ in range(12)]
K_nom = argmin(lambda K: cost(*K))
K_dr = argmin(lambda K: np.mean([cost(*K, **e)
                                 for e in medii]))
```

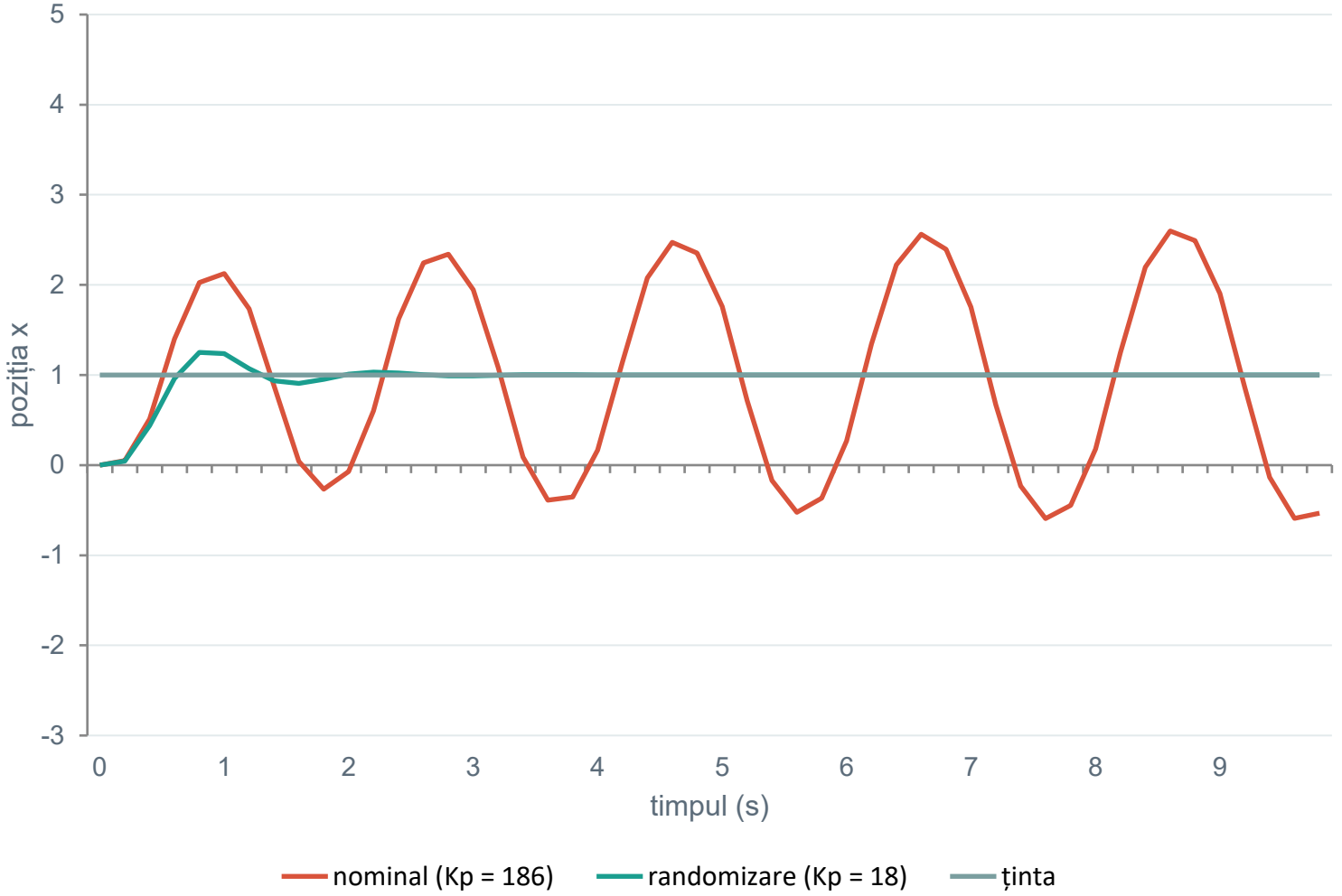
- Euler explicit cu pas 0,02 s (S7)
- Întârzierea: o coadă FIFO de comenzi
- Câștigurile randomizate minimizează costul **mediu** pe cele 12 medii
- Testăm apoi pe sistemul real și pe 200 de medii noi

Rezultatele: optim în simulare nu înseamnă optim în realitate

Controler	Kp, Kd	Cost în simulare	Cost pe sistemul real	Eșecuri (200 de medii)
Reglat pe nominal	186, 24	**0,173**	11,56	55%
Reglat cu randomizare	18, 6	0,238	**0,384**	**9,5%**

- Controlerul nominal este agresiv (câștiguri mari): perfect fără întârziere, instabil cu întârziere
- Controlerul robust este mai lent în simulator (cost cu 38% mai mare), dar de 30 de ori mai bun pe sistemul real
- Eșec = cost peste 1, adică sistemul oscilează sau nu ajunge la țintă

Pe sistemul real: controlerul nominal oscilează



- Nominal: oscilații din ce în ce mai mari, limitate doar de saturarea motorului (axa e tăiată la [−3; 5])
- Randomizare: o depășire moderată, apoi stabilizare
- Cauza: întârzierea de 0,12 s pe care simulatorul nominal nu o avea
- Un câștig mare este optim doar dacă modelul este **exact**

Prețul și câștigul robusteții

55% / 9,5%

medii noi în care controlerul **eșuează**: nominal / randomizare

+38%

cost **în simulatorul nominal** pentru controlerul robust – prețul plătit

30×

cost mai mic **pe sistemul real** pentru controlerul robust

- Optimizarea pe un singur model este o formă de **supraînvățare** – pe simulator, nu pe date
- Aceeași lecție ca la polinomul de grad 12: performanța pe datele de antrenare nu spune nimic despre lumea reală

Robotica: totul învățat în simulare

- **OpenAI, 2019**: o mână robotică rezolvă cubul Rubik, cu politica antrenată **numai** în simulare
- Randomizare automată a mărimii cubului, frecării, maselor, întârzierilor – echivalentul a mii de ani de experiență
- **Roboți patrupezi** (ETH Zürich, Isaac Gym, 2021): mii de roboți în paralel învață să meargă în câteva minute pe GPU
- Politicile trec apoi direct pe roboții reali: scări, zăpadă, teren accidentat
- Roboții umanoizi actuali se antrenează la fel: simulare masiv paralelă + randomizare + puține date reale

Rezultat surprinzător

Mâna OpenAI a rezistat perturbărilor pe care nu le văzuse niciodată (o mănușă, degete legate, o girafă de pluș care o împinge) – efect al randomizării largi.

Mașinile autonome: miliarde de mile în simulare

- Waymo: aproape **200 de milioane de mile** autonome pe drumuri publice și **miliarde** de mile în simulare
- Scenariile reale sunt **reluate** și modificate: alt pieton, altă viteză, altă vreme – mii de variante dintr-un incident
- Evenimentele periculoase se testează doar în simulare: nu putem provoca accidente pe stradă
- Date de senzori **sintetice** (cameră, lidar) generate de modele ale lumii (Waymo World Model, 2026, pe baza Genie 3)
- Validarea simulatorului devine parte a argumentului de siguranță în fața autorităților

S **Argumentul statistic**

Pentru a demonstra prin rulaj real că o mașină autonomă este mai sigură decât un șofer uman ar fi nevoie de sute de milioane de mile (RAND, 2016). Simularea completează, dar nu înlocuiește, acest argument.

Riscurile antrenării pe lumi simulate

- **Exploatarea simulatorului**: agentul găsește erori ale fizicii simulate (sărituri prin pereți, energie din nimic)
- **Recompensă prost specificată**: agentul face exact ce i-am cerut, nu ce am vrut – un clasic al învățării prin întărire
- **Prăbușirea modelelor**: modelele antrenate repetat pe date generate de alte modele își pierd diversitatea (Shumailov et al., Nature 2024)
- **Încredere falsă**: un simulator nevalidat produce un sistem care pare sigur doar în simulare
- Toate se reduc la aceeași întrebare din S6: cât de **valid** este modelul pentru scopul în care îl folosim?

V **Regula**

Un simulator folosit pentru antrenarea AI este un model ca oricare altul: trebuie verificat, validat și documentat, cu domeniul lui de valabilitate.

Verificare

? Controlerul reglat pe simulatorul nominal are costul 0,173 în simulare și 11,6 pe sistemul real. Care este cauza principală?

A zgomotul numeric al metodei Euler

B controlerul a exploatat un simulator fără întârziere; câștigurile mari devin instabile cu întârziere

C grila de căutare pentru Kp și Kd a fost prea rară

D costul a fost calculat greșit pe sistemul real

Ce reținem din ora a doua

- ✓ Modelele meteo AI sunt **surogate** la scară planetară: mii de ori mai ieftine, dar construite pe date produse de fizică
- ✓ PINN pune **ecuația în funcția de cost**: 6 măsurători + fizică dau curba completă și parametrul k
- ✓ SINDy descoperă **formula** din date, dacă biblioteca conține termenii potriviți
- ✓ Simularea antrenează AI; **randomizarea domeniului** face politicile robuste la decalajul sim-to-real

Ora 3

Gemeni digitali și sinteza cursului

1. Ce este (și ce nu este) un geamăn digital
2. Asimilarea datelor: filtrul Kalman
3. Demo: geamănul unei camere de servere
4. Sinteza celor 10 săptămâni

Ce este un geamăn digital

Un model virtual legat continuu de sistemul real

Un set de construcții informaționale virtuale care imită **structura**, **contextul** și **comportamentul** unui sistem natural, tehnic sau social, este **actualizat dinamic** cu date de la geamănul fizic, are **capacitate predictivă** și **informează decizii** care produc valoare.

National Academies of Sciences, Engineering, and Medicine – Foundational Research Gaps and Future Directions for Digital Twins, 2024

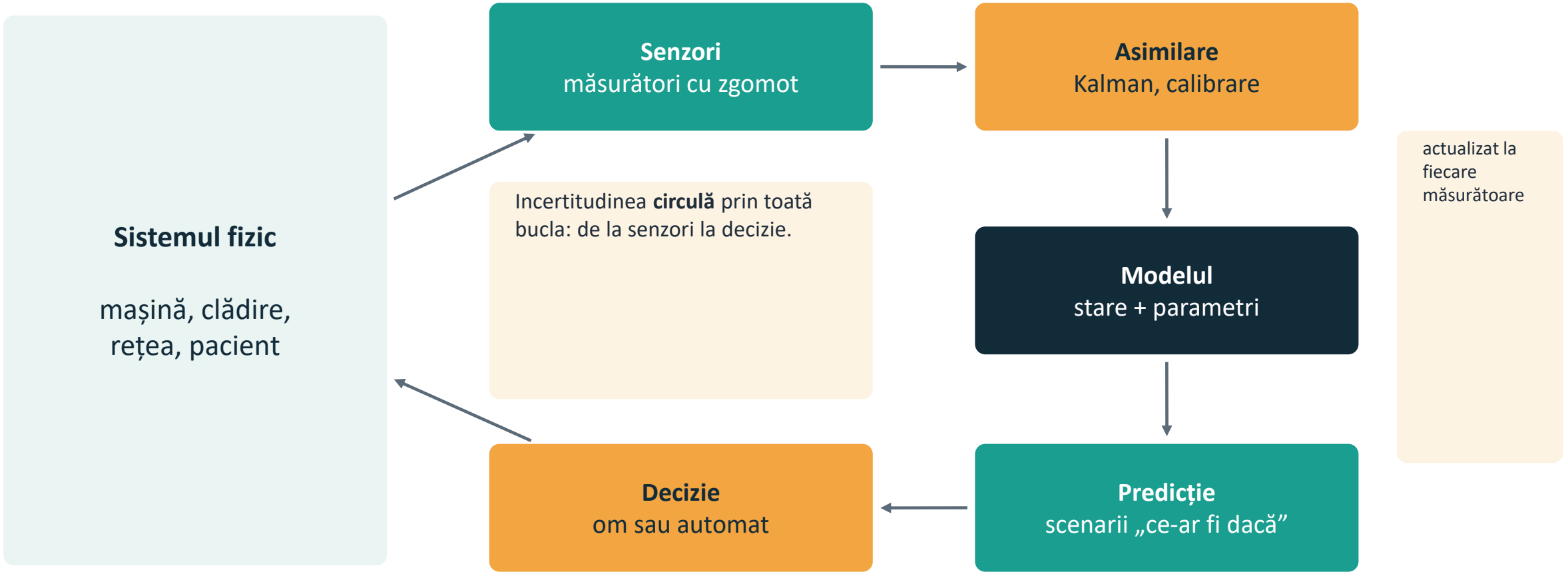
- **Interacțiunea bidirecțională** dintre virtual și fizic este esențială (același raport)
- Fizic ↦ virtual: senzorii actualizează starea și parametrii modelului
- Virtual ↦ fizic: predicțiile și deciziile modifică sistemul real
- Fără predicție și decizie, avem doar o vizualizare a datelor

Model digital, umbră digitală, geamăn digital

	Model digital	Umbră digitală	Geamăn digital
Fizic ↦ virtual	manual, o singură dată	**automat**, continuu	**automat**, continuu
Virtual ↦ fizic	manual	manual	**automat** sau asistat
Exemplu	simularea urgențelor din S5	tablou de bord cu starea liniei de producție	termostat care învață clădirea și reglează încălzirea
Ce lipsește	actualizarea cu date	bucla de decizie	–

- Clasificare după Kritzinger et al. (2018); în practică, multe produse numite „geamăn digital” sunt de fapt umbre digitale
- Toate modelele din S1–S9 au fost modele digitale: calibrate o dată, apoi folosite

Bucla bidirecțională a unui geamăn digital



Componentele unui geamăn digital – și unde le-am învățat

1 Modelul

Mecanist (S4–S9), învățat sau hibrid (S10). Trebuie să ruleze mai repede decât timpul real.

2 Datele

Senzori, jurnale, evenimente; calitate, frecvență, lipsuri (S6).

3 Asimilarea

Combină modelul cu măsurătorile: filtrul Kalman, calibrare continuă (S10).

4 Predicția și incertitudinea

Replicări, intervale, ansambluri (S2, S6); surogate pentru viteză (S10).

5 Decizia

Scenarii comparate cu numere aleatoare comune, optimizare (S6).

6 VVUQ continuu

Verificare, validare și cuantificarea incertitudinii – repetate cât timp sistemul se schimbă.

Un geamăn digital nu este o tehnologie nouă, ci o combinație disciplinată de metode pe care le cunoașteți deja – cu o singură piesă nouă, asimilarea datelor.

De unde vine ideea

- **Apollo 13** (1970): simulatoarele de la sol, legate de datele navei, au ajutat la găsirea soluției de întoarcere
- **Grieves** (2002): conceptul de model virtual pereche al unui produs, pe tot ciclul de viață
- **NASA** (2010): termenul „digital twin” în foaia de parcurs tehnologică – vehicule monitorizate pe tot zborul
- **Industrie 4.0** (din 2011): senzori ieftini, cloud, IoT – geamănul devine posibil pentru orice mașină
- **Azi**: de la turbine și motoare de avion la orașe, pacienți și Pământul întreg

Atenție la etichetă

O vizualizare 3D a unei fabrici cu date în timp real nu este un geamăn digital dacă nu are un model care prezice și o buclă de decizie.

Piesa nouă

Asimilarea datelor: filtrul Kalman

1. Modelul deviază, senzorul are zgomot
2. Pasul de predicție și pasul de corecție
3. Câștigul Kalman: pe cine credem
4. Extensii: EKF, stare augmentată, ansambluri

Nici modelul, nici senzorul nu ajung singure

Doar modelul

- neted, fără zgomot
- dar **deviază**: parametri greșiți, perturbări
- nu observă schimbările din sistem
- eroarea crește în timp
- exemplu: răcirea se degradează, modelul nu știe

vs

Doar senzorul

- urmărește realitatea
- dar are **zgomot** și uneori lipsește
- măsoară doar o parte din stare
- nu prezice viitorul
- exemplu: temperatura, nu și eficiența răcirii

Asimilarea datelor combină cele două: modelul prezice, măsurătoarea corectează, iar ponderea fiecăruia depinde de incertitudinea lui. Bonus: putem estima mărimi care nu se măsoară direct, ca eficiența răcirii.

Filtrul Kalman (1): predicția cu modelul

$$\hat{x}^{-}(t) = f(\hat{x}(t-1)) \qquad P^{-}(t) = F \cdot P(t-1) \cdot F^{T} + Q$$

$\hat{x}$	estimarea stării: aici temperatura T și eficiența răcirii k
f, F	modelul (un pas Euler, S7) și jacobianul lui – matricea derivatelor parțiale
P	matricea de covarianță: **cât de nesiguri** suntem pe fiecare componentă
Q	zgomotul de proces: cât greșește modelul la un pas; mare = modelul e slab

Predicția avansează starea cu modelul și **crește** incertitudinea: P^{-} este mai mare decât P, pentru că se adaugă Q. Fără măsurători, incertitudinea ar crește la nesfârșit – exact ce se întâmplă cu un model simplu lăsat să ruleze.

Filtrul Kalman (2): corecția cu măsurătoarea

$$K = P^{-}H^{T}(HP^{-}H^{T} + R)^{-1} \qquad \hat{x} = \hat{x}^{-} + K\cdot(y - H\hat{x}^{-}) \qquad P = (I - KH)\cdot P^{-}$$

y	măsurătoarea; H spune ce parte a stării se măsoară (aici doar T)
$y - H\hat{x}^{-}$	**inovația**: cât de surprinși suntem de măsurătoare
R	zgomotul senzorului ($\sigma^2 = 0{,}25$); mare = senzorul e slab
K	**câștigul Kalman**: ce fracțiune din surpriză acceptăm

Corecția **scade** incertitudinea. Prin covarianța dintre T și k, o surpriză în temperatură corectează și eficiența răcirii, deși k nu este măsurat niciodată direct.

Intuiția într-o singură dimensiune

$$\hat{x} = \hat{x}^{-} + K \cdot (y - \hat{x}^{-}), \quad K = P^{-} / (P^{-} + R)$$

$K \approx 0$	senzor foarte zgomotos ($R \gg P^{-}$): **ignorăm** măsurătoarea, credem modelul
$K \approx 1$	model foarte nesigur ($P^{-} \gg R$): **copiem** măsurătoarea
medie ponderată	$\hat{x} = (1 - K) \cdot \hat{x}^{-} + K \cdot y$: ponderile sunt **invers proporționale** cu dispersiile

Exemplu: modelul prezice 35 °C cu dispersia 1, senzorul arată 37 °C cu dispersia 0,25. K = 1 / 1,25 = 0,8, deci estimarea este 35 + 0,8·2 = 36,6 °C, cu dispersia (1 − 0,8)·1 = 0,2 – mai mică decât a oricărei surse.

Dincolo de filtrul liniar

- **EKF** (Kalman extins): pentru modele neliniare liniarizăm la fiecare pas cu jacobianul F – ce folosim în demo
- **Stare augmentată**: un parametru necunoscut (k) devine componentă a stării, cu o mică plimbare aleatoare (S3)
- **Kalman cu ansambluri** (EnKF): covarianța estimată din zeci de rulări ale modelului – standard în meteorologie
- **Filtre cu particule**: mii de simulări ponderate după cât de bine explică datele – merg și pentru agenți (S9)
- **4D-Var**: ajustăm starea inițială ca traiectoria modelului să treacă prin toate observațiile unei ferestre

K

Kalman, 1960

Filtrul a ajuns rapid în navigația programului Apollo. Azi rulează în fiecare telefon (GPS + accelerometre), dronă, avion și sistem de prognoză meteo.

Demo

Geamănul digital al unei camere de servere

1. Modelul termic și scenariul de degradare
2. Filtrul Kalman extins pe temperatură și eficiență
3. Detectarea degradării
4. Decizia: „ce-ar fi dacă” reducem sarcina?

Modelul camerei de servere

Element	Valoare
Ecuația (bilanț termic, S7)	$C \cdot dT/dt = P - k \cdot (T - T_a)$: căldura produsă minus căldura extrasă de răcire
Parametri	capacitatea termică C = 50, aerul rece T_a = 18 °C, puterea serverelor P = 60
Răcirea	k = 4 (nominal); după **minutul 180** scade liniar până la **2,4** la minutul 240 (filtru înfundat)
Senzorul	o măsurătoare pe minut, zgomot cu σ = 0,5 °C; durata: 6 ore
Echilibrul	$T = T_a + P/k$: **33 °C** cu k = 4, **43 °C** cu k = 2,4; limita de alarmă: 40 °C

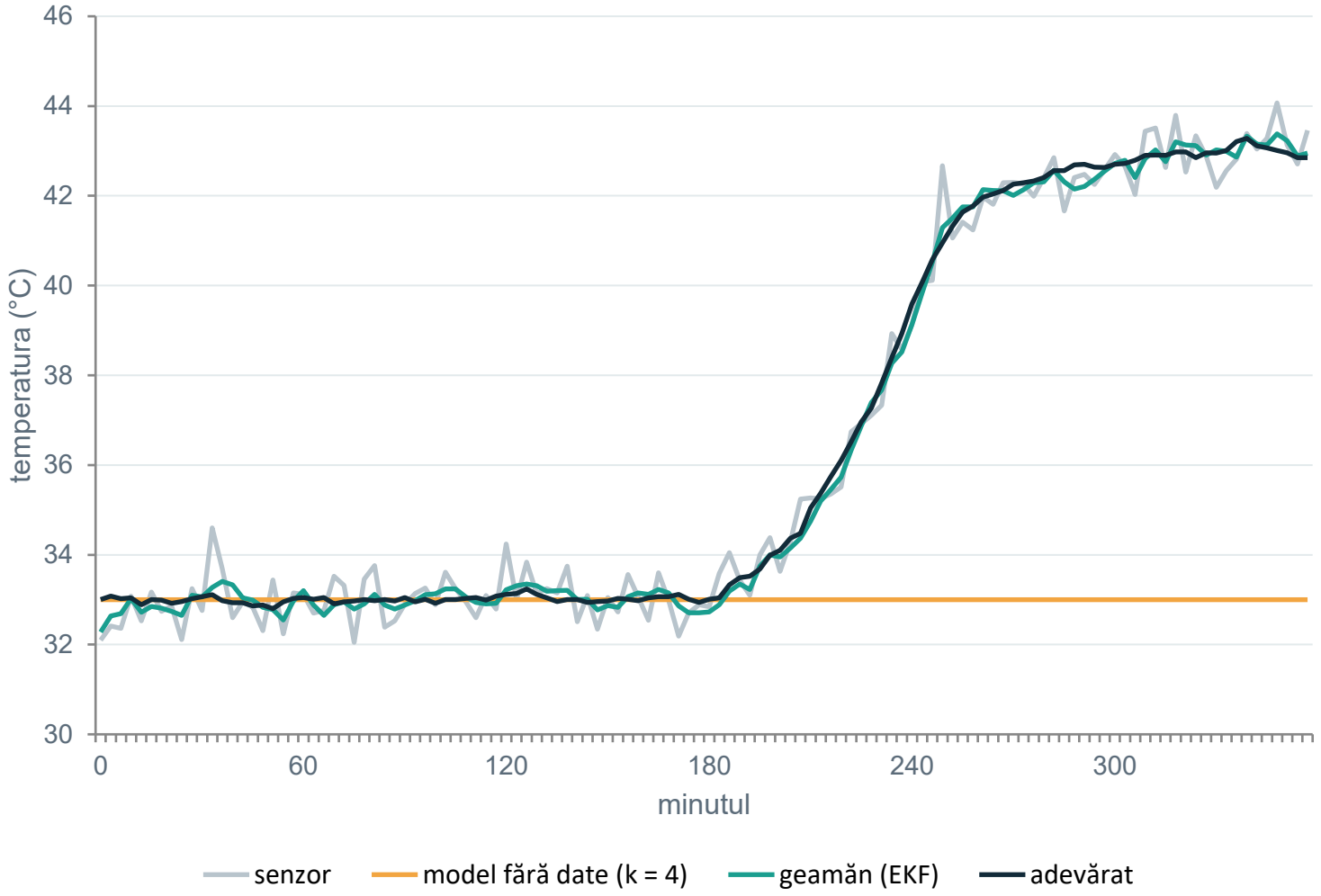
- Modelul „fără date” presupune k = 4 pentru totdeauna – ca un model digital calibrat o singură dată
- Geamănul estimează **simultan** temperatura și eficiența k, din măsurătorile de temperatură

Filtrul Kalman extins cu stare augmentată [T, k]

```
x = np.array([33.0, 4.0])              # [T, k]
P = np.diag([1.0, 0.5]); Q = np.diag([0.05**2, 0.004])
R = 0.5**2; H = np.array([1.0, 0.0])
for t in range(360):                   # minute
    # predicție: un pas Euler + jacobianul
    T, k = x
    x = np.array([T + (Pw - k*(T - Ta)) / C, k])
    F = np.array([[1 - k/C, -(T - Ta)/C], [0, 1]])
    P = F @ P @ F.T + Q
    # corecție cu măsurătoarea y[t]
    K = P @ H / (H @ P @ H + R)
    x = x + K * (y[t] - x[0])
    P = P - np.outer(K, H) @ P
```

- k nu se măsoară: are în Q o mică variație (plimbare aleatoare), ca să poată urmări schimbările
- F: derivatele pasului Euler în raport cu T și k
- Câștigul K are două componente: corectează și T, și k
- Fiecare pas: câteva operații cu matrice 2 × 2

Temperatura: modelul fără date, senzorul și geamănul



- Până la minutul 180 toate trei coincid
- După degradare, modelul fără date rămâne la 33 °C – **nu știe** nimic
- Senzorul urmărește creșterea, dar cu zgomot de ±0,5 °C
- Geamănul urmărește valoarea adevărată, fără zgomot

Eroarea fiecărei surse de informație

7,92 °C

RMSE al **modelului fără date**, după degradare

0,48 °C

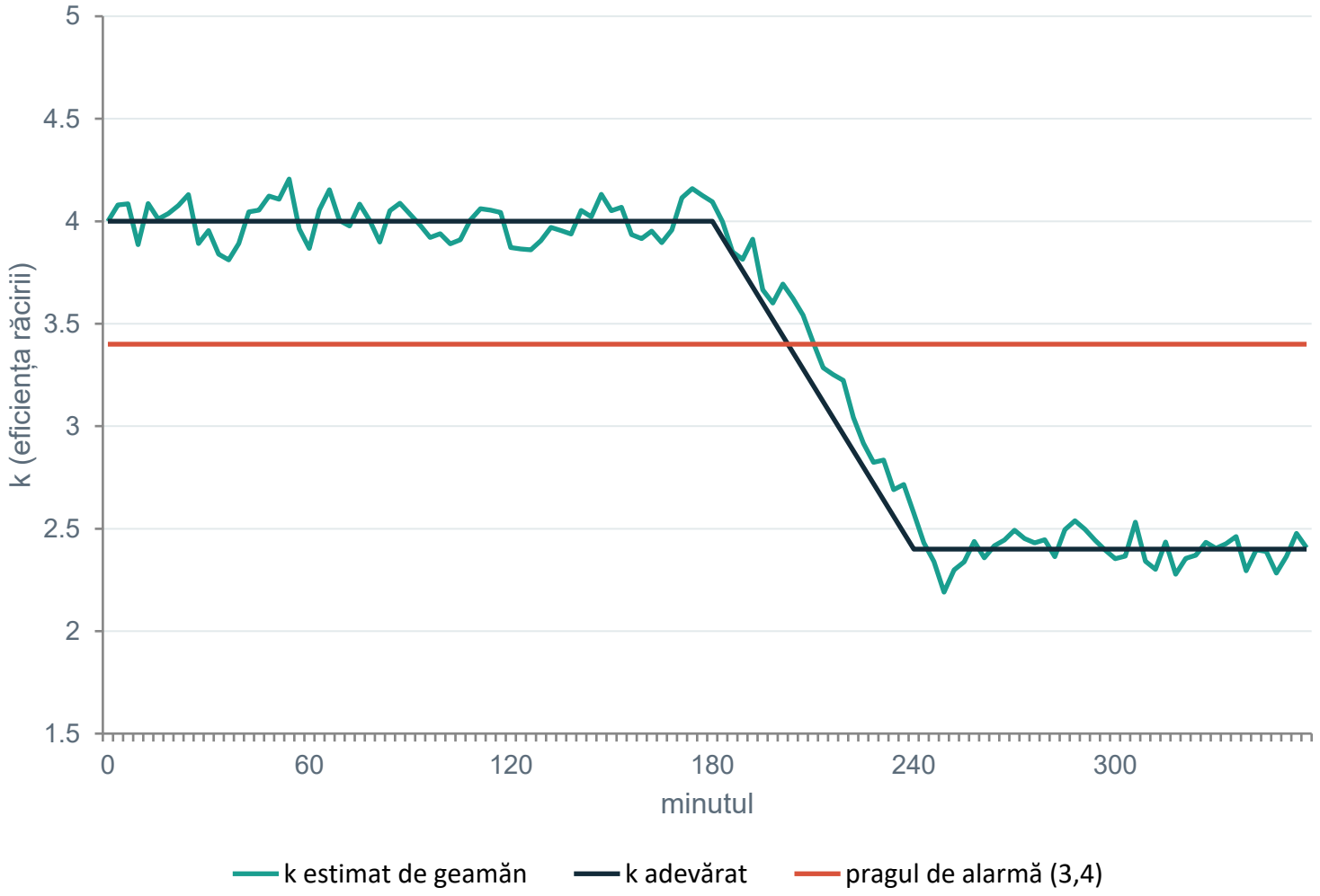
RMSE al **senzorului** (zgomotul lui)

0,20 °C

RMSE al **geamănului** (EKF) – mai bun decât ambele

- Geamănul are o eroare de 2,4 ori mai mică decât senzorul și de 40 de ori mai mică decât modelul fără date
- Același principiu ca în exemplul 1D: combinarea a două surse nesigure dă o estimare mai bună decât oricare dintre ele

Eficiența răcirii: nemăsurată, dar estimată



- k nu este măsurat niciodată – este dedus din felul în care **evoluează** temperatura
- Estimarea are zgomot, dar urmărește degradarea cu o mică întârziere
- Alarmă când k estimat scade sub 3,4 (85% din nominal): **minutul 211**
- La final: k = 2,58 ± 0,15 (adevărat 2,4)

Ce aflăm din geamăn, înainte de problemă

min. 211

alarma de degradare: la 31 de minute după începutul ei, cu temperatura încă sub limită

2,58 ± 0,15

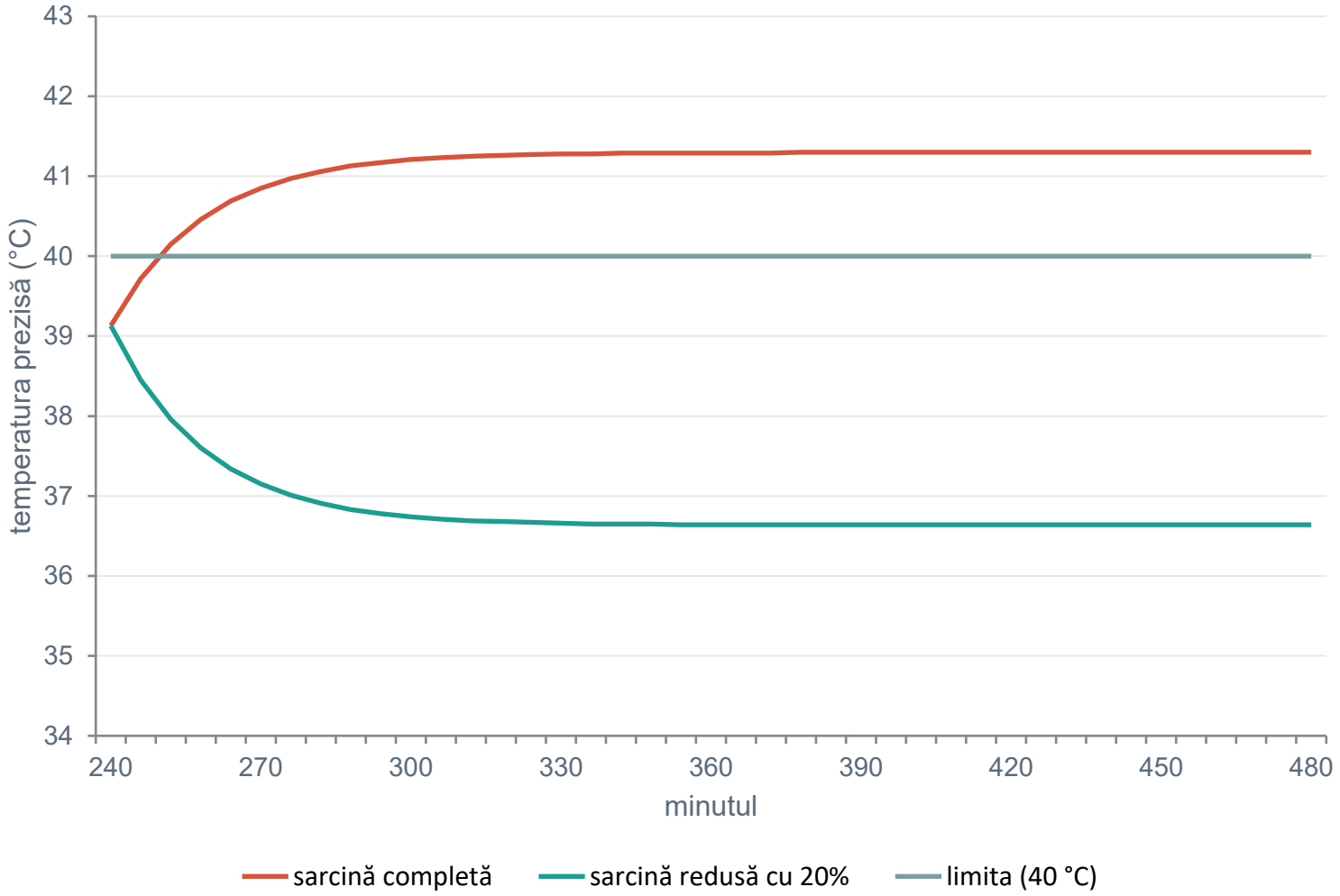
eficiența estimată la final, **cu incertitudine** (adevărat: 2,4)

43 °C

echilibrul prezis cu noua eficiență – **peste limita de 40 °C**

- Un senzor singur ar fi declanșat alarma abia la depășirea limitei; geamănul o anunță pe baza **cauzei** (răcirea), nu a efectului
- Mentenanță predictivă: schimbăm filtrul înainte ca serverele să se supraîncălzească

Decizia la minutul 240: ce-ar fi dacă reducem sarcina cu 20%?



- Starea curentă (din geamăn): T = 39,1 °C, k ≈ 2,58
- Sarcină completă: depășește 40 °C în **10 minute**, echilibru 41,3 °C
- Sarcină redusă: rămâne sub limită, echilibru **36,6 °C**
- Recomandare: mutăm 20% din sarcină și programăm intervenția

Bucla se închide: de la predicție la acțiune

- Decizia (mutarea sarcinii) **modifică** sistemul fizic: P scade de la 60 la 48
- Geamănul continuă să asimileze date și **verifică** dacă predicția s-a adeverit
- Dacă temperatura nu scade cum s-a prezis, modelul sau estimarea lui k sunt greșite – o nouă alarmă
- Predicția presupune k constant; dacă degradarea continuă, scenariul real este **mai rău** – incertitudinea trebuie comunicată
- Decizia finală o poate lua un om (asistat) sau sistemul (automat), după cât de critic este riscul

€ Valoarea

Fără geamăn: supraîncălzire, servere oprite de protecție, intervenție de urgență. Cu geamăn: 20% din sarcină mutată pentru câteva ore și un filtru schimbat la timp.

Verificare

? Înlocuim senzorul cu unul de **4 ori mai zgomotos** (σ = 2 °C, deci R de 16 ori mai mare). Ce face filtrul Kalman?

A câștigul K crește: credem mai mult senzorul

B câștigul K scade: credem mai mult modelul, estimarea e mai netedă, dar reacționează mai lent

C nimic: filtrul elimină orice zgomot

D filtrul nu mai poate estima k deloc

Dincolo de demo

Gemeni digitali în practică

1. Industrie, energie, sănătate, orașe
2. Destination Earth: geamănul Pământului
3. VVUQ continuu și provocări
4. Când nu merită un geamăn digital

Unde se folosesc gemenii digitali

1 Fabrici

Linii de producție simulate în paralel cu cele reale: blocaje, mentenanță, reconfigurări testate virtual (S5).

2 Motoare de avion și turbine

Fiecare motor are modelul lui, actualizat cu datele din zbor: mentenanță la nevoie, nu la calendar.

3 Rețele electrice

Echilibrarea producției regenerabile și a consumului; scenarii de avarie în timp real.

4 Sănătate

Modele ale inimii unui pacient, construite din imagini, pentru planificarea intervențiilor.

5 Orașe

Trafic, energie, inundații: agenți (S9) și automate celulare (S8) alimentate cu date din oraș.

6 Centre de date

Răcire și plasarea sarcinii, ca în demo – economii de energie de ordinul procentelor.

Numitorul comun: un sistem scump sau critic, date continue de la senzori și decizii repetate în care o predicție bună valorează bani sau vieți.

Destination Earth: un geamăn digital al Pământului

- Inițiativa Comisiei Europene, cu ECMWF, ESA și EUMETSAT; primele sisteme lansate în **iunie 2024**
- **Geamănul extremelor meteo**: simulări la rezoluție de câțiva kilometri pentru inundații, valuri de căldură, furtuni
- **Geamănul adaptării la schimbările climatice**: proiecții pe decenii, pentru decizii în energie, agricultură, urbanism
- Rulează pe supercalculatoarele EuroHPC (S8): LUMI, Leonardo, MareNostrum 5
- Întrebări „ce-ar fi dacă”: ce se întâmplă cu o regiune la +2 °C? Unde construim diguri?

E **Toate piesele cursului**

Ecuații continue (S7), HPC (S8), ansambluri și incertitudine (S2, S6), asimilarea datelor și surogate AI (S10) – într-un singur sistem.

VVUQ continuu și provocările gemenilor digitali

- **Validarea expiră**: sistemul fizic se schimbă (uzură, reconfigurări), deci validarea din S6 trebuie **repetată** continuu
- **Incertitudinea** trebuie propagată până la decizie: un singur număr prezis nu ajunge pentru o decizie critică
- **Datele**: senzori defecți, lipsuri, formate incompatibile – o mare parte din efortul real
- **Securitate și confidențialitate**: geamănul unui pacient sau al unei rețele electrice este o țintă
- **Costul**: modele, senzori, infrastructură și oameni – trebuie justificat de valoarea deciziilor

National Academies, 2024

Raportul identifică VVUQ, asimilarea datelor la scară mare și modelele hibride rapide drept principalele lacune de cercetare.

Când nu merită un geamăn digital

- Când o **decizie unică** se poate lua cu un studiu de simulare clasic (S5, S6) – fără flux continuu de date
- Când nu există **senzori** sau datele nu reflectă starea care contează
- Când nu există **decizii** de luat pe baza predicțiilor – rămâne o vizualizare scumpă
- Când modelul nu poate rula **mai repede** decât sistemul real și nu avem un surogat
- Când un simplu **prag pe un senzor** rezolvă problema la fel de bine

M

Începeți mic

Un model simplu, un senzor bine ales, un filtru Kalman și o singură decizie – ca în demo. Complexitatea se adaugă doar când valoarea o justifică.

Verificare

? Care dintre următoarele este un **geamăn digital** în sensul definiției National Academies?

A un model 3D detaliat al unei fabrici, folosit la prezentări

B o simulare a urgențelor calibrată anul trecut pe datele din registru

C un model actualizat automat din senzori, care prezice defecțiunile și programează mentenanța

D un tablou de bord care afișează în timp real valorile senzorilor

Încheiere

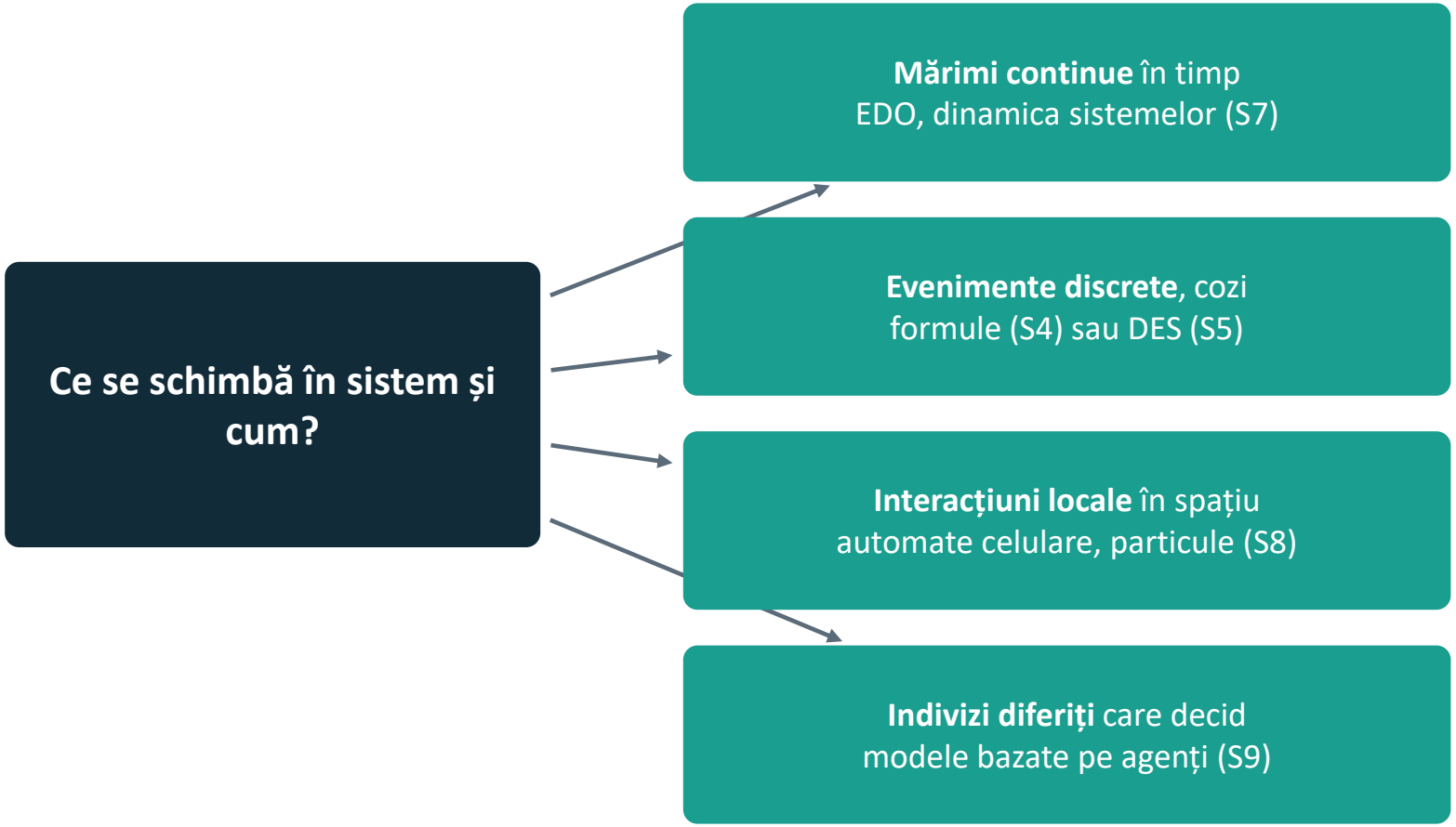
Sinteza cursului

1. Cele 10 săptămâni într-un tabel
2. Cum alegem metoda potrivită
3. Formulele care trebuie știute
4. Principiile care se repetă

Cele 10 săptămâni ale cursului

S	Tema	Ideea de reținut
1	Sisteme, modele, simulare	ciclul de modelare: întrebare, model, verificare, validare, experimente
2	Aleatorul și metoda Monte Carlo	generatori și transformarea inversă; eroarea scade ca $1/\sqrt{n}$
3	Procese stocastice, lanțuri Markov	starea de echilibru $\pi P = \pi$; plimbări aleatoare
4	Teoria cozilor	Little $L = \lambda W$; M/M/1, Pollaczek–Khinchine; variabilitatea costă
5	Simularea cu evenimente discrete	ceasul sare de la eveniment la eveniment; lista de evenimente, SimPy
6	Date de intrare, rezultate, validare	replicări și intervale de încredere; verificare ≠ validare
7	Sisteme dinamice continue	EDO, Euler / RK4, stabilitate, rigiditate; SIR, Lotka–Volterra
8	Automate celulare, particule, HPC	reguli locale, comportament global; Verlet; Amdahl
9	Agenți și sisteme complexe	emergență, rețele, ODD, calibrare ABC
10	Date, AI + simulare, gemeni digitali	surogate, PINN, sim-to-real, filtrul Kalman

Cum alegem metoda: un arbore de decizie



Intrări nesigure? **Monte Carlo** și replicări (S2, S6) – pentru orice ramură.

Simularea e prea lentă? **Surogat** (S10).

Date în timp real și decizii repetate? **Geamăn digital** (S10).

Formulele care trebuie știute

Ce calculăm	Formula	Săptămâna
Eroarea Monte Carlo	$\sigma / \sqrt{n}$	S2
Legea lui Little	$L = \lambda \cdot W$	S4
Coada M/M/1	$\rho = \lambda/\mu,\ W = 1/(\mu - \lambda)$	S4
Pollaczek–Khinchine	$Wq = \rho \cdot (1 + Cs^2)/(2\mu(1 - \rho))$	S4
Interval de încredere	$\bar{x} \pm t \cdot s/\sqrt{n}$	S6
Replicări necesare	$n = (t \cdot s/h)^2$	S6
Pasul Euler	$y(t + h) = y(t) + h \cdot f(t, y)$	S7
Numărul de reproducere	$R_0 = \beta/\gamma$	S7
Legea lui Amdahl	$S(N) = 1 / ((1 - p) + p/N)$	S8
Regresie, câștigul Kalman	$R^2 = 1 - \Sigma(y - \hat{y})^2/\Sigma(y - \bar{y})^2,\ K = P^-/(P^- + R)$	S10

Principiile care au revenit în fiecare săptămână

- **Întâi întrebarea**, apoi modelul: același sistem are modele diferite pentru întrebări diferite
- **Cel mai simplu model** care răspunde la întrebare – complexitatea se adaugă doar când e nevoie
- **O rulare nu înseamnă nimic**: replicări, intervale de încredere, incertitudine
- **Verificare și validare**: codul face ce credem? Modelul seamănă cu realitatea pentru scopul nostru?
- **Verificați numerele**: am găsit erori în aproape fiecare material sursă – recalculați cu un program
- **Reproductibilitate**: sămânță fixă, cod publicat, documentare (ODD) – altcineva trebuie să obțină același rezultat

Pregătirea pentru examen

- Examen cu **întrebări grilă și răspunsuri numerice scurte**, corectate automat – ca întrebările „Verificare”
- Întrebările de verificare din toate cele 10 suporturi de curs sunt un bun antrenament (cu răspunsurile explicate)
- Exersați calculele numerice: Little, M/M/1, P-K, intervale de încredere, un pas Euler, Amdahl, R^2
- Înțelegeți **când** se folosește fiecare metodă și ce presupuneri face – nu doar formulele
- Notebook-urile demo: rulați-le, schimbați parametrii, prevedeți rezultatul înainte să îl vedeți

✓ **Sfat**

Pentru fiecare formulă, întrebați-vă ce presupuneri face și ce se întâmplă când nu sunt îndeplinite. Aceste întrebări apar cel mai des.

Verificare

? O urgență are sosiri care variază pe ore, priorități, timpi de consult lognormali și 3 sau 4 medici. Vrem timpul mediu de așteptare cu un interval de încredere. Ce alegem?

A formula M/M/c, fără simulare

B un model cu ecuații diferențiale (S7)

C simulare cu evenimente discrete, cu replicări independente

D un singur run lung al unui model bazat pe agenți

Rezumatul săptămânii

- ✓ Regresia și modelele învățate: evaluate **numai** pe date nefolosite; supraînvățarea apare pe date și pe simulatoare
- ✓ Surogatele (LHS + proces gaussian) înlocuiesc mii de simulări cu câteva zeci – calibrare în secunde, nu în ore
- ✓ Fizica + datele: PINN, SINDy, modele meteo AI; simularea antrenează AI, randomizarea domeniului o face robustă
- ✓ Geamănul digital: model + date + asimilare (Kalman) + predicție + decizie, într-o buclă bidirecțională

Ce urmează după acest curs

- **Unelte**: SimPy și Mesa (Python), AnyLogic (multi-metodă), NetLogo, Modelica (sisteme fizice), JAX (simulare diferențiabilă)
- **Cursuri înrudite**: învățare automată, cercetări operaționale, calcul paralel, sisteme de control, statistică bayesiană
- **Proiecte**: alegeți un sistem pe care îl cunoașteți (o cantină, un server, o intersecție) și duceți-l prin tot ciclul
- **Lucrare de licență**: surogate, gemeni digitali și simularea pentru AI sunt teme căutate, cu multe direcții deschise
- **Cariere**: inginerie, logistică, finanțe, sănătate, jocuri, meteorologie, robotică – peste tot unde decizii costă

Final de curs

Mulțumesc!

- Toate modelele sunt greșite, dar unele sunt utile (G. E. P. Box)
- Un model bun răspunde la o întrebare, nu imită totul
- Verificați, validați, replicați – și spuneți cât de sigur sunteți

Construim modele, le simulăm, le punem la îndoială. Toate trei contează – mai ales ultima.

Bibliografie pentru săptămâna 10

- Ș. Holban – Tehnici de modelare și simulare, cap. 8: modele corelaționale (materiale de curs); datele London bike sharing (laboratorul 7)
- J. P. C. Kleijnen – Design and Analysis of Simulation Experiments, 2015; Rasmussen, Williams – Gaussian Processes for ML, 2006
- M. Raissi, P. Perdikaris, G. E. Karniadakis – Physics-informed neural networks, J. Comput. Phys. 378, 2019; Z. Li et al. – Fourier Neural Operator, ICLR 2021
- S. L. Brunton, J. L. Proctor, J. N. Kutz – Discovering governing equations from data (SINDy), PNAS 113, 2016
- R. Lam et al. – GraphCast, Science 2023; I. Price et al. – GenCast, Nature 2024; D. Kochkov et al. – NeuralGCM, Nature 2024; ECMWF – AIFS, 2025
- J. Tobin et al. – Domain randomization for transferring deep neural networks from simulation to the real world, IROS 2017; OpenAI – Solving Rubik's Cube with a robot hand, 2019
- R. E. Kalman – A new approach to linear filtering and prediction problems, 1960; National Academies – Foundational Research Gaps and Future Directions for Digital Twins, 2024
- W. Kritzinger et al. – Digital Twin in manufacturing: a categorical literature review and classification, IFAC 2018
- Prezentare realizata cu ajutorul Claude Fable 5.1 și Claude Opus 5.5